18-740: Modern Computer Architecture & Design

Lecture 7:
"Dealing with Long-Latency Memory Instructions"

Olivia Hsu

September 15, 2026

- Required Reading Assignment:
  - **Chapter 3 of Shen and Lipasti (SnL).**
- **Lab 2** handout posted on Canvas today (9/15/2026)
- **HW 1** due 9/18/2026
- Recommended References:
  - Chapter 2 & Appendix B of Hennessey and Patterson 5th Edition.



Electrical & Computer ENGINEERING

18-740: Modern Computer Architecture & Design

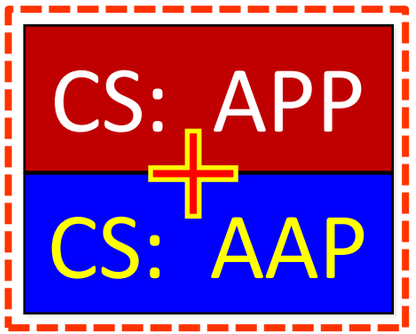
Lecture 7:
"Dealing with Long-Latency Memory Instructions"

A. Virtual Memory Revisited (18-213/18-613)
B. Memory Operations and Dataflow
C. Modern Superscalar Out-of-Order Processors (ARM CPUs)
D. Snapdragon 8 Gen 2 + ARM Cortex A715 & X3 CPU Cores

Electrical & Computer ENGINEERING

A. Virtual Memory Revisited (18-213/18-613)

"Computer Systems: A Programmer's Perspective" (CS:APP)
by Randy Bryant & Dave O'Hallaron



A Programmer's Perspective of Computer Systems

An Architect's Perspective of Computer Systems

Electrical & Computer ENGINEERING

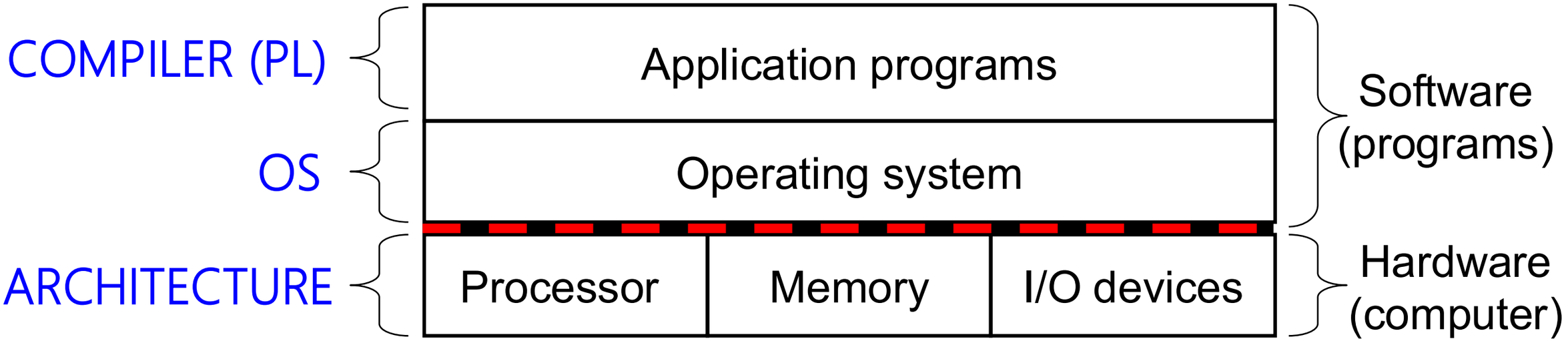
Anatomy of a Computer System: SW/HW

➢ What is a Computer System?

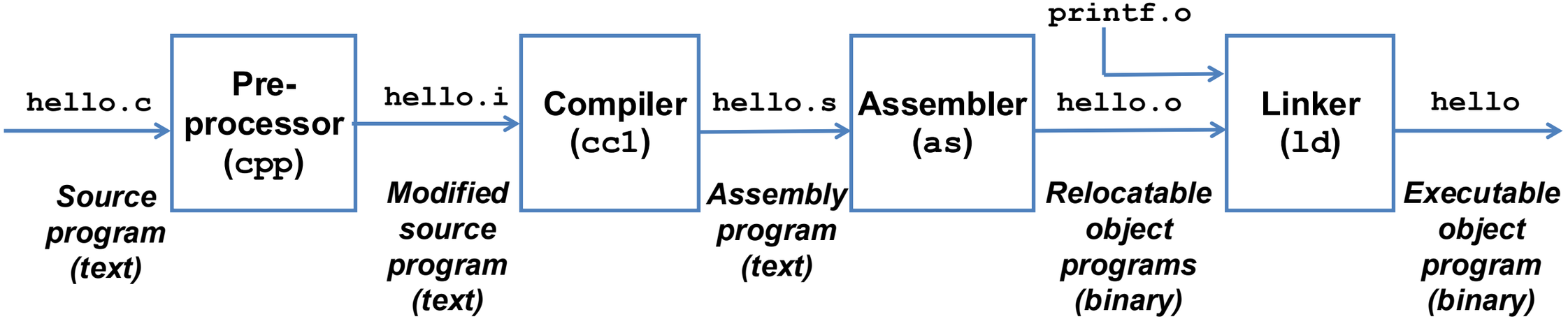
- ❖ = [Software + Hardware] = [Programs + Computer]

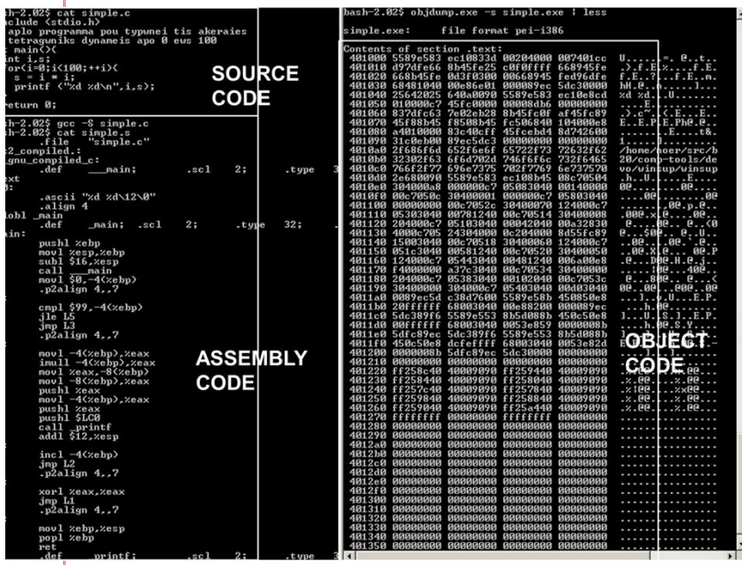
  = [Application program + OS] + [Processor + Memory + I/O Devices]

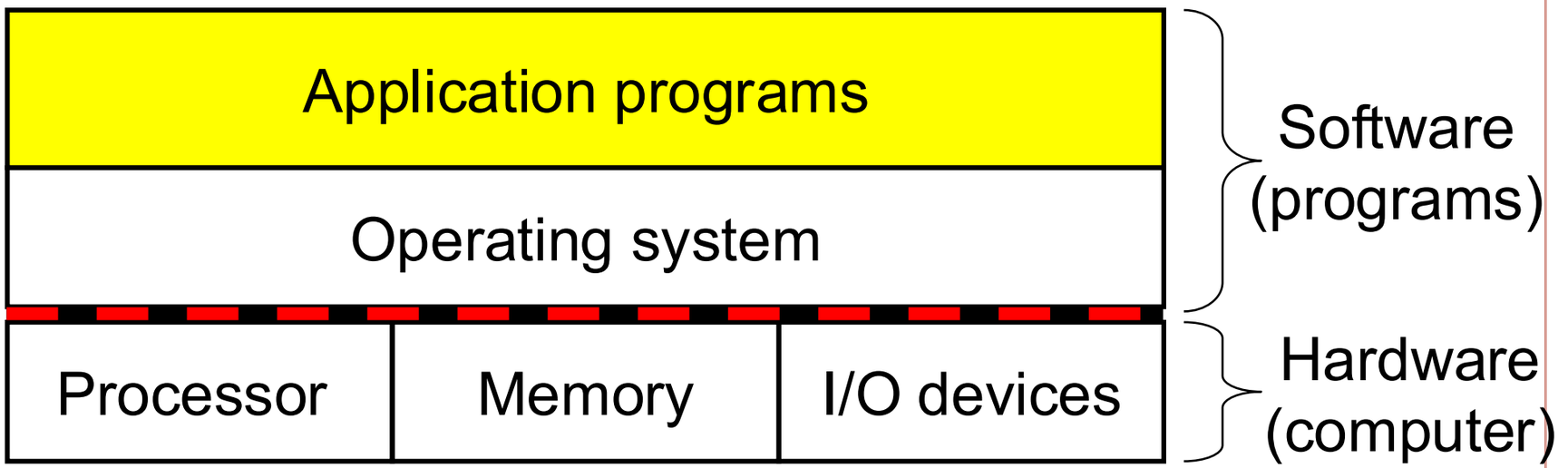
- ❖ **Programming Languages + Operating Systems + Computer Architecture**



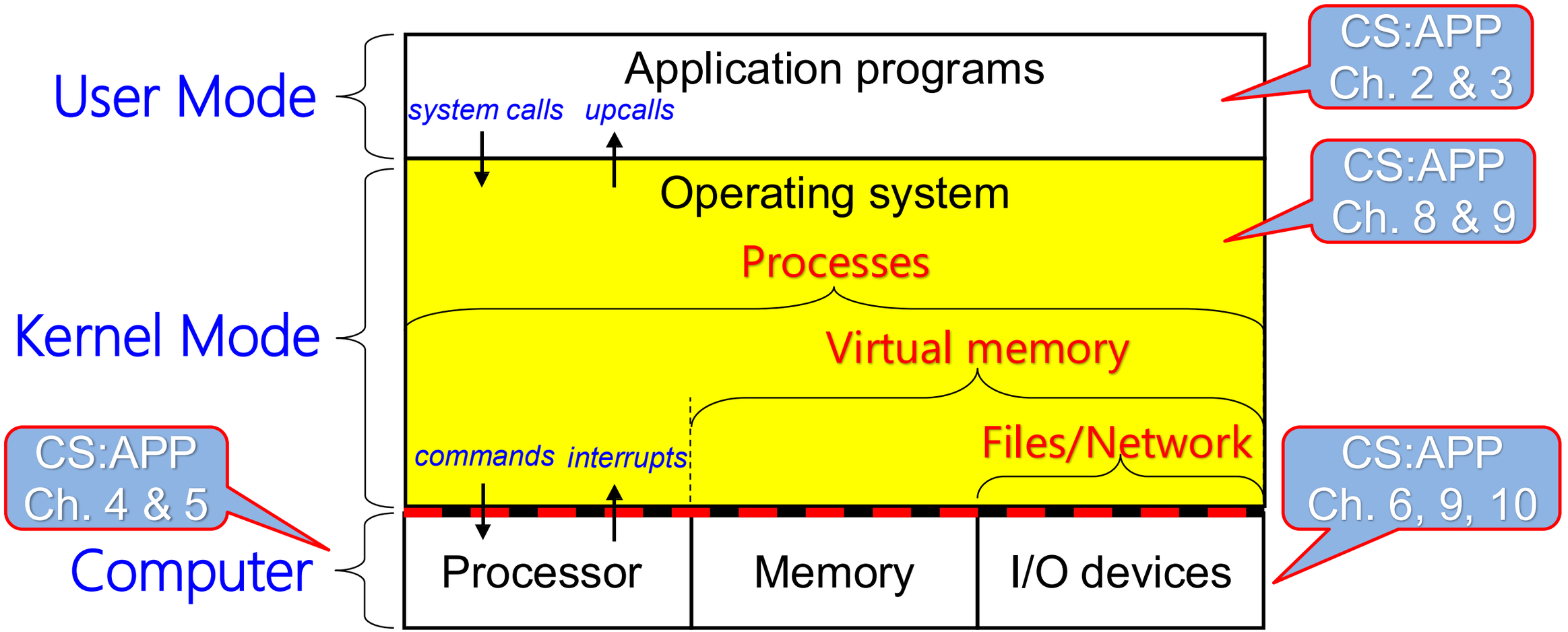
Anatomy of a Computer System: Compiler







Anatomy of a Computer System: OS



Operating System Abstractions

Abstraction 1: Processes

application:	***program***
OS:	***process***
hardware:	***processor***

Abstraction 2: Virtual memory

application:	***address space***
OS:	***virtual memory***
hardware:	***physical memory***

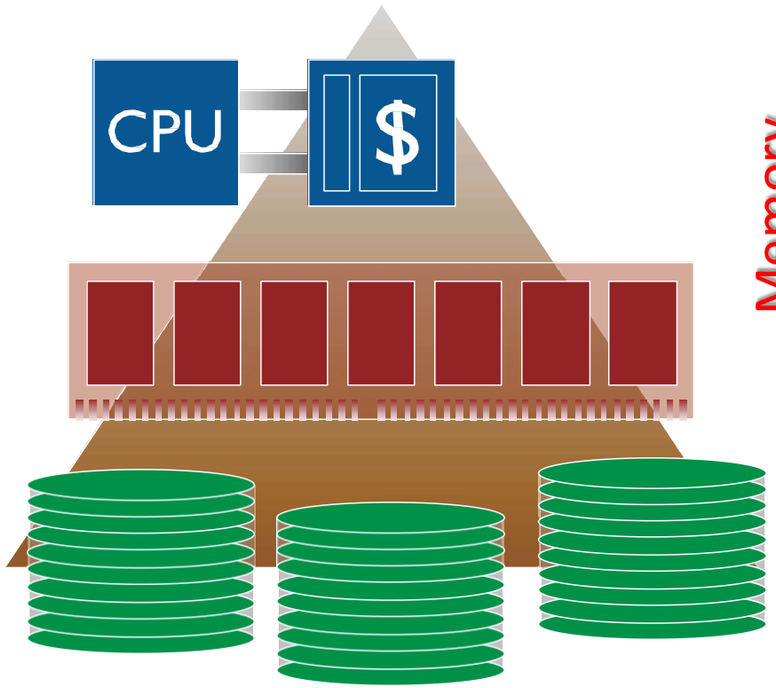
Abstraction 3: File System

application:	***copy file1 file2***
OS:	***files, directories***
hardware:	***Storage: disk/SSD***

Abstraction 4: Messaging

application:	***sockets***
OS:	***TCP/IP protocols***
hardware:	***network interface***

Memory Hierarchy: Main Memory Abstraction



Memory Abstraction

Register File
32 words, sub-nsec

L1 cache (SRAM)
~32 KB, ~nsec

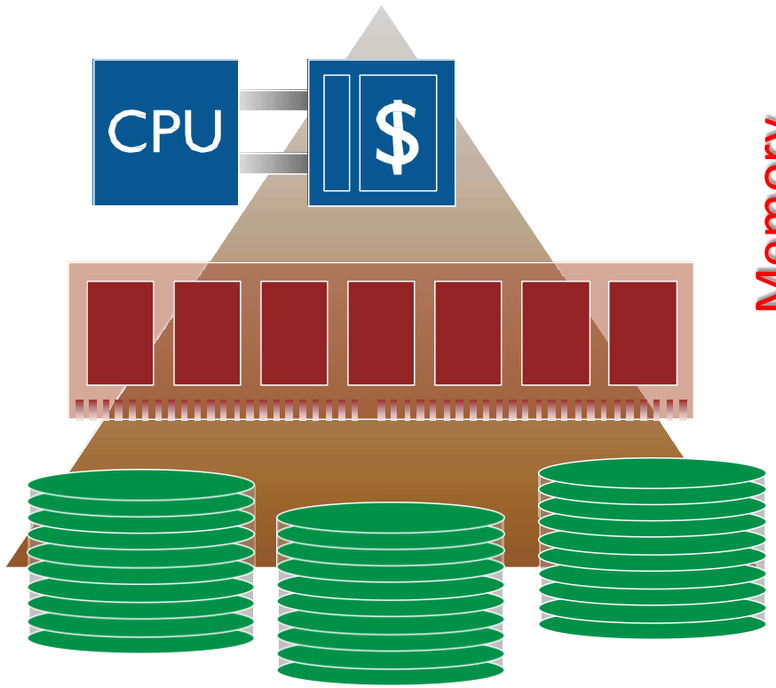
L2 cache (SRAM)
512 KB, ~ 1 MB, many nsec

L3 cache, (SRAM)
4 MB, 8 MB, ...

Main Memory (DRAM)
4-16 GB, ~100 nsec

Disk Storage or Flash SSD (mobile)
1K-1000K GB, ~10 msec

Memory Hierarchy: Multi-Level Cache Memories



Memory Abstraction

Register File
32 words, sub-nsec

L1 cache (SRAM)
~32 KB, ~nsec

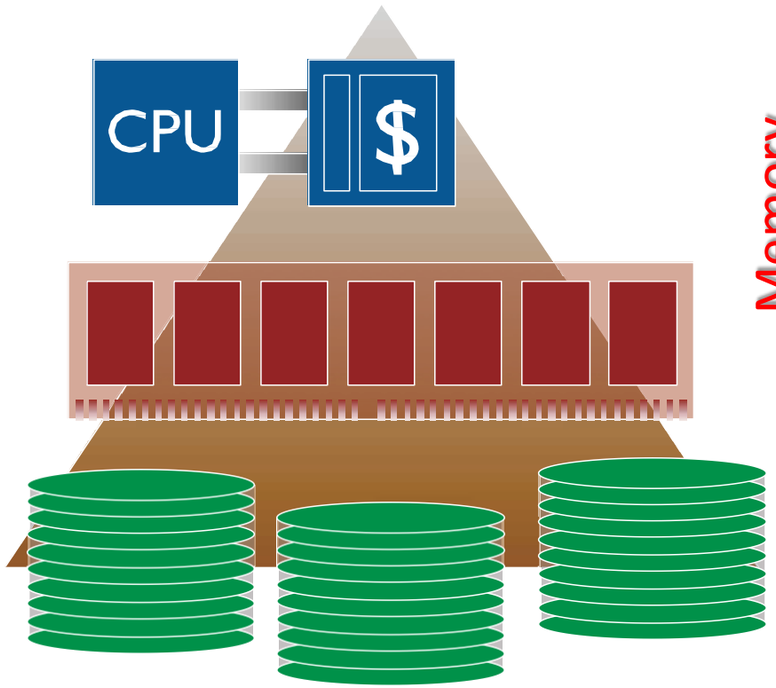
L2 cache (SRAM)
512 KB, ~ 1 MB, many nsec

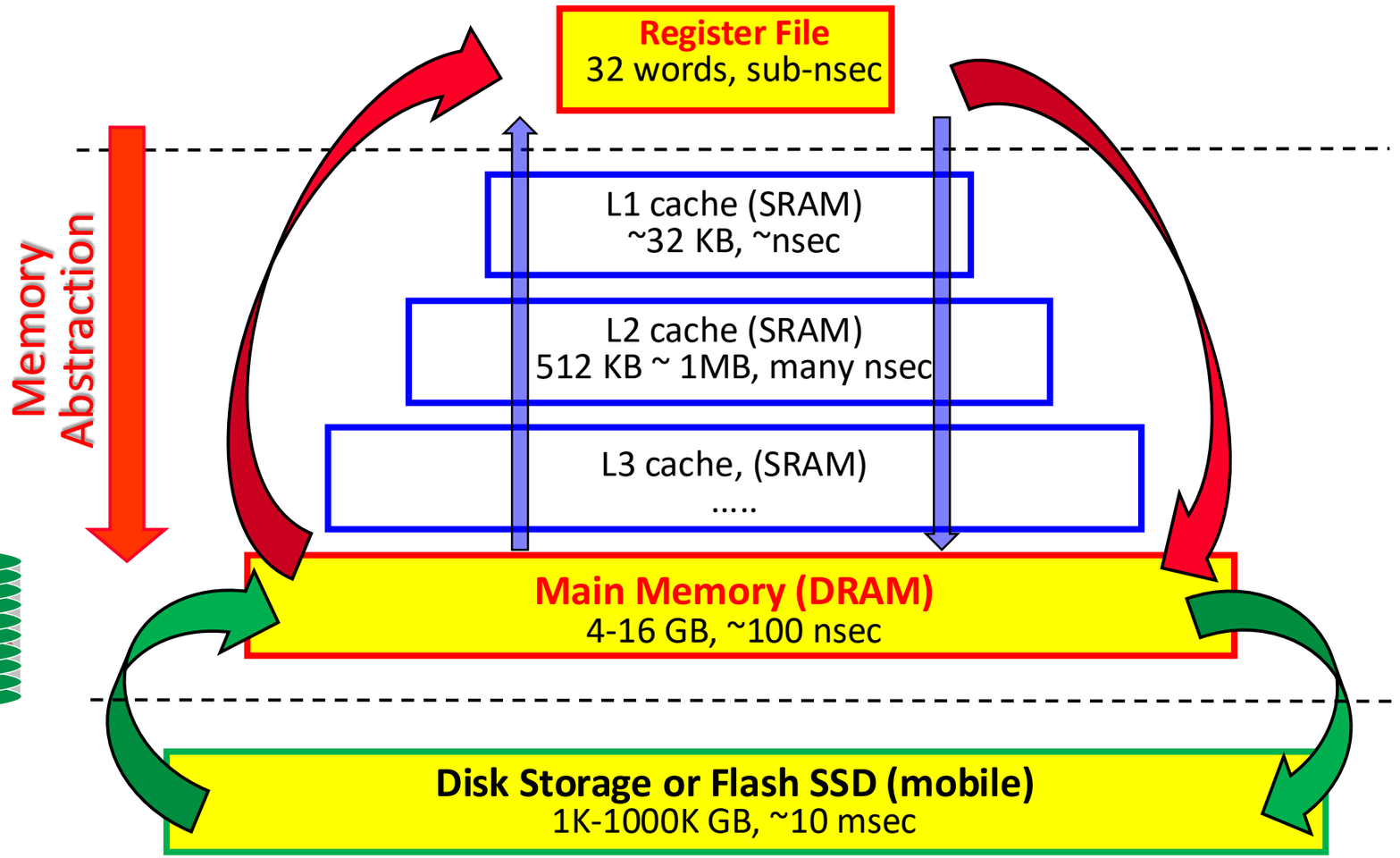
L3 cache, (SRAM)
4 MB, 8 MB, ...

Main Memory (DRAM)
4-16 GB, ~100 nsec

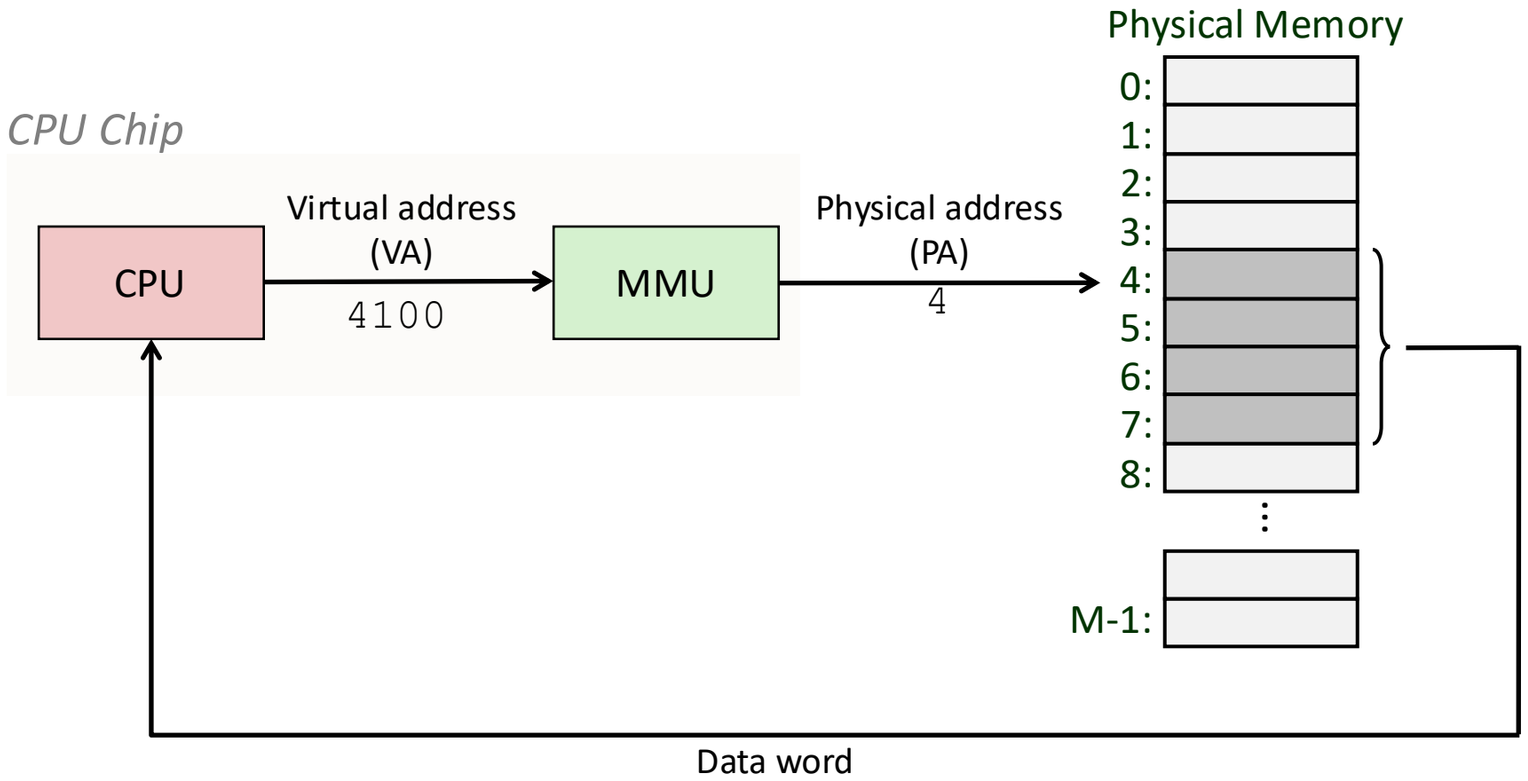
Disk Storage or Flash SSD (mobile)
1K-1000K GB, ~10 msec

Memory Hierarchy: Virtual Memory (OS managed)





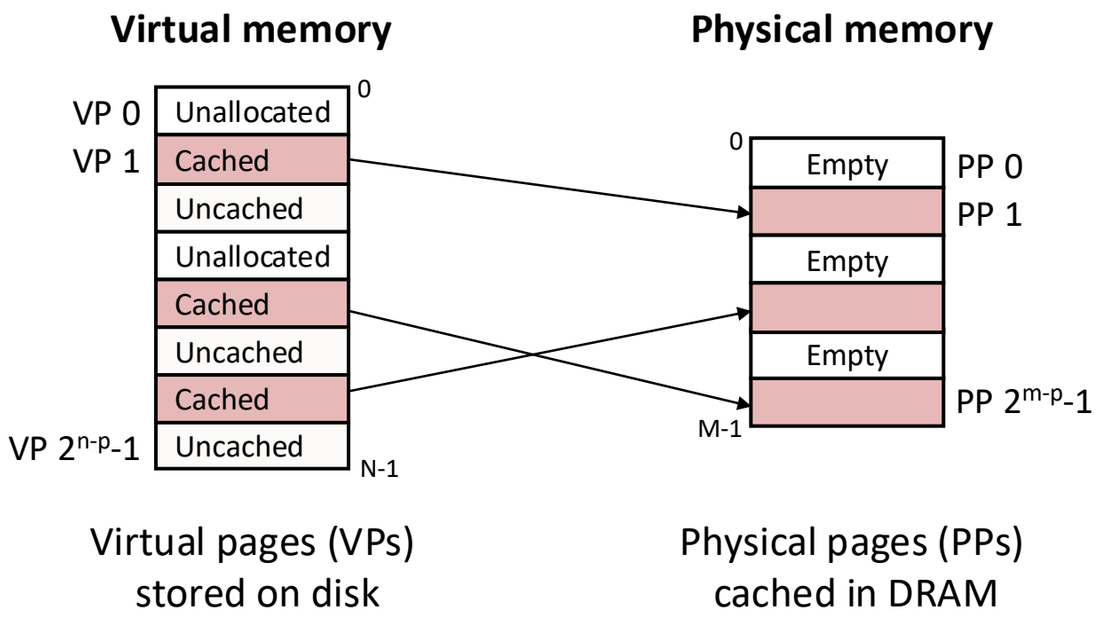
A System Using Virtual Addressing



- Used in all modern servers, laptops, and smart phones
- One of the great ideas in computer science

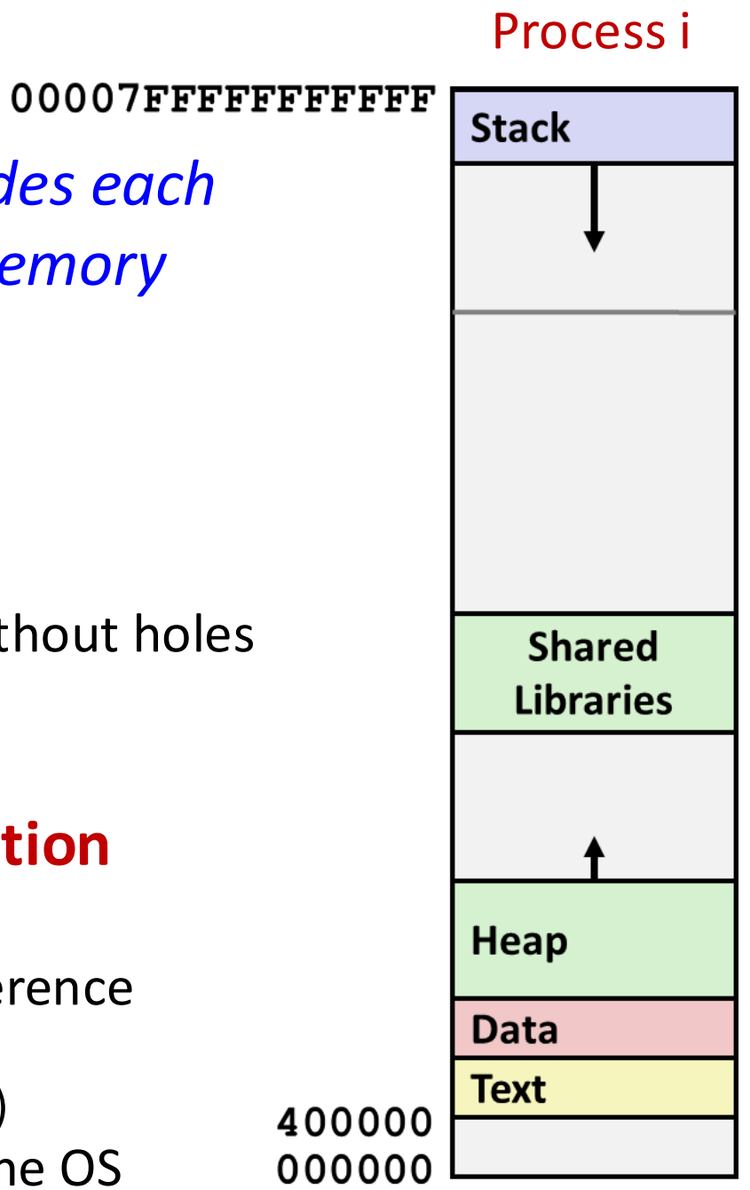
Virtual Memory as a Tool for Caching Pages

- Conceptually, *virtual memory* is an array of N contiguous bytes (addressable by the program) usually stored on persistent storage.
- The contents of the array in storage are cached in *physical memory* (*DRAM*)
  - These cached blocks are called ***pages*** (size is P = 2^p bytes)



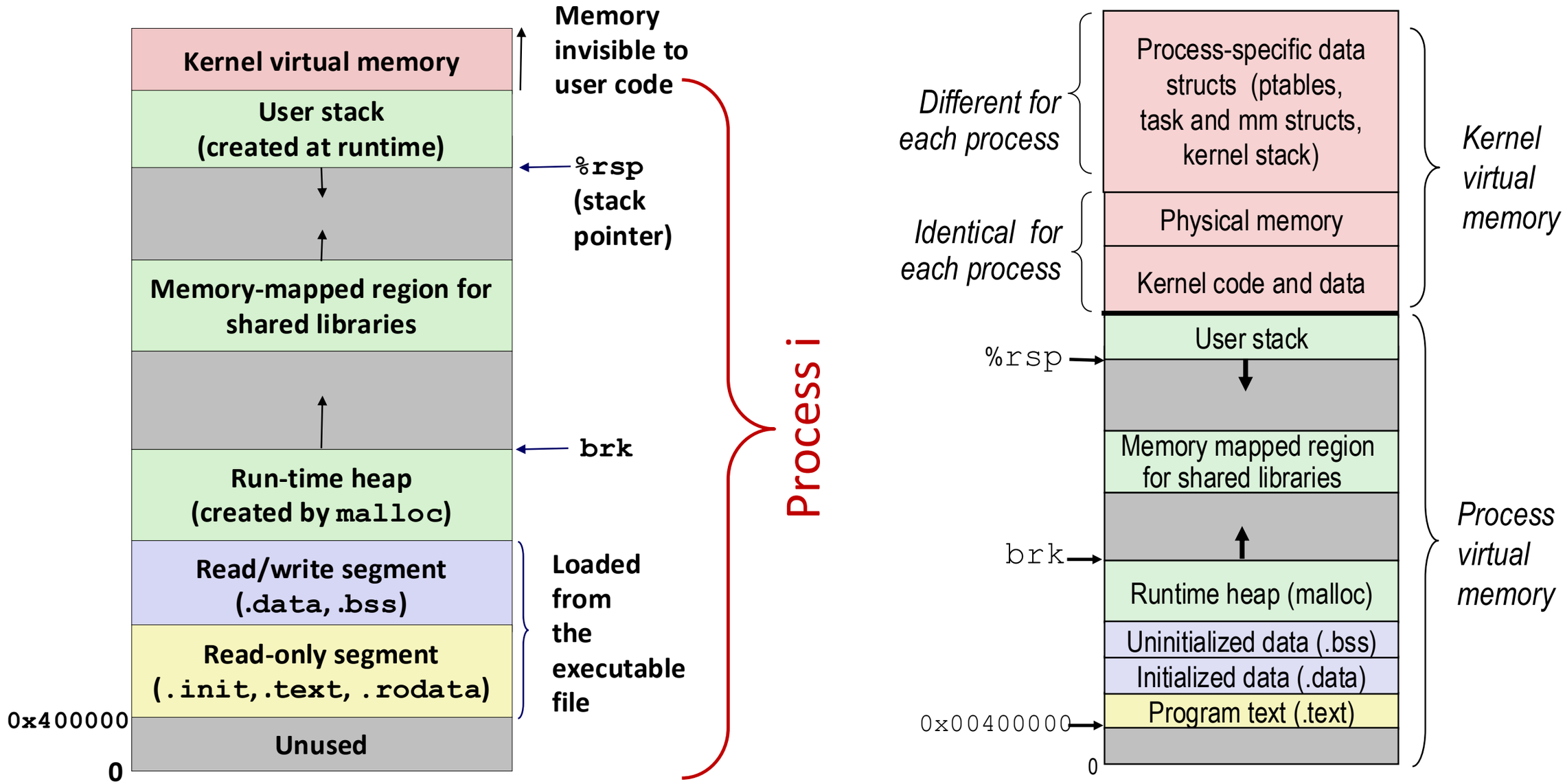
Why (Page-Based) Virtual Memory?

Modern Virtual Memory



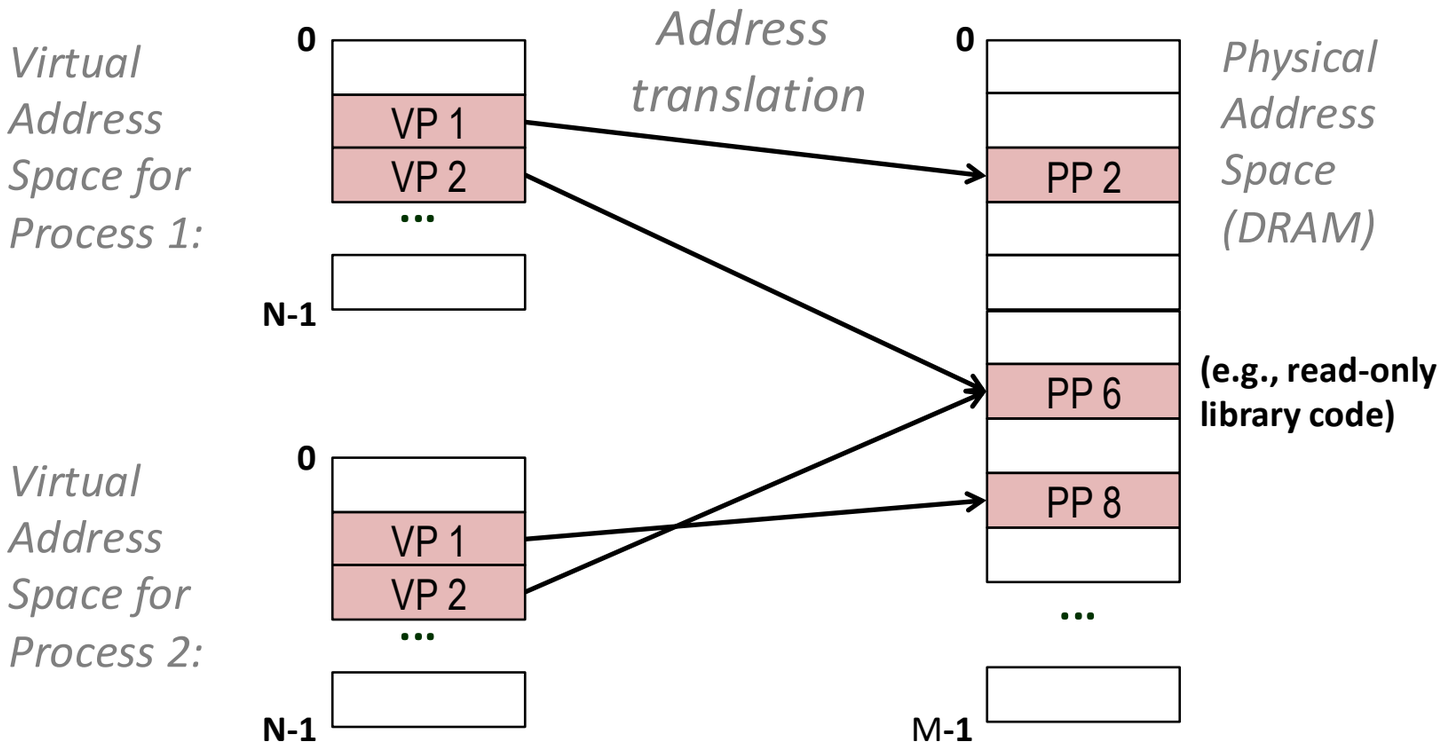
- ➢ *In a multi-tasking system, Virtual Memory (VM) provides each process with the illusion of a large, private, uniform memory*
- VM for Memory Management (Demand Paging)
  - Capacity of secondary memory (swap space on disk)
  - At the speed of primary memory (DRAM)
- VM for Memory Protection (Permission Bits)
  - Each process sees a large, contiguous memory segment without holes
  - Each process's memory space is private,
    i.e. protected from access by other processes
- Based on a common HW mechanism: **Address Translation**
  - User process operates on "virtual" or "effective" addresses
  - HW translates from virtual to physical address on each reference
    - Controls which physical locations can be named by a process
    - Allows dynamic relocation of physical backing store (DRAM vs. HD)
  - VM HW and memory management policies controlled by the OS

Virtual Address Space of a Linux Process



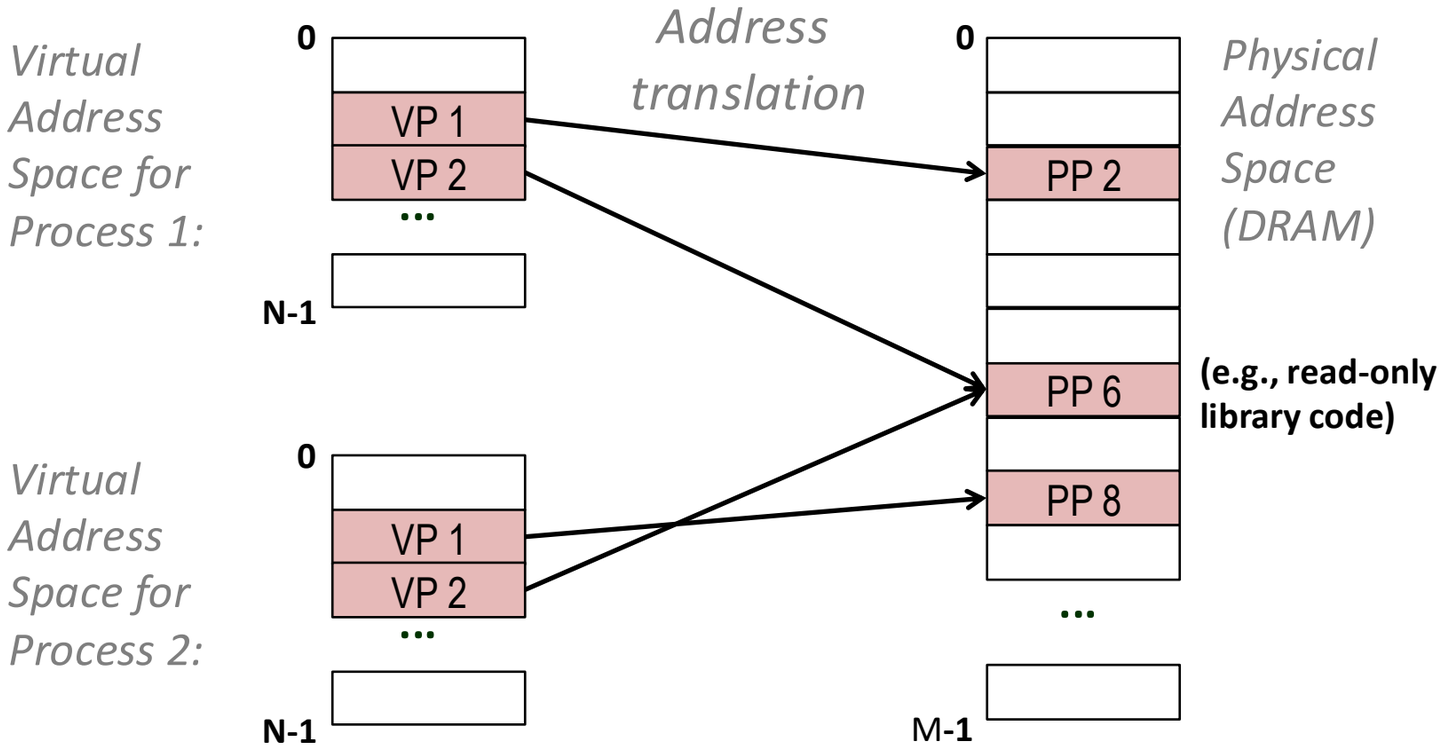
VM as a Tool for Memory Management

- Key idea: each process has its own virtual address space
  - It can view memory as a simple linear array
  - Mapping function scatters addresses through physical memory
    - Well-chosen mappings can improve locality



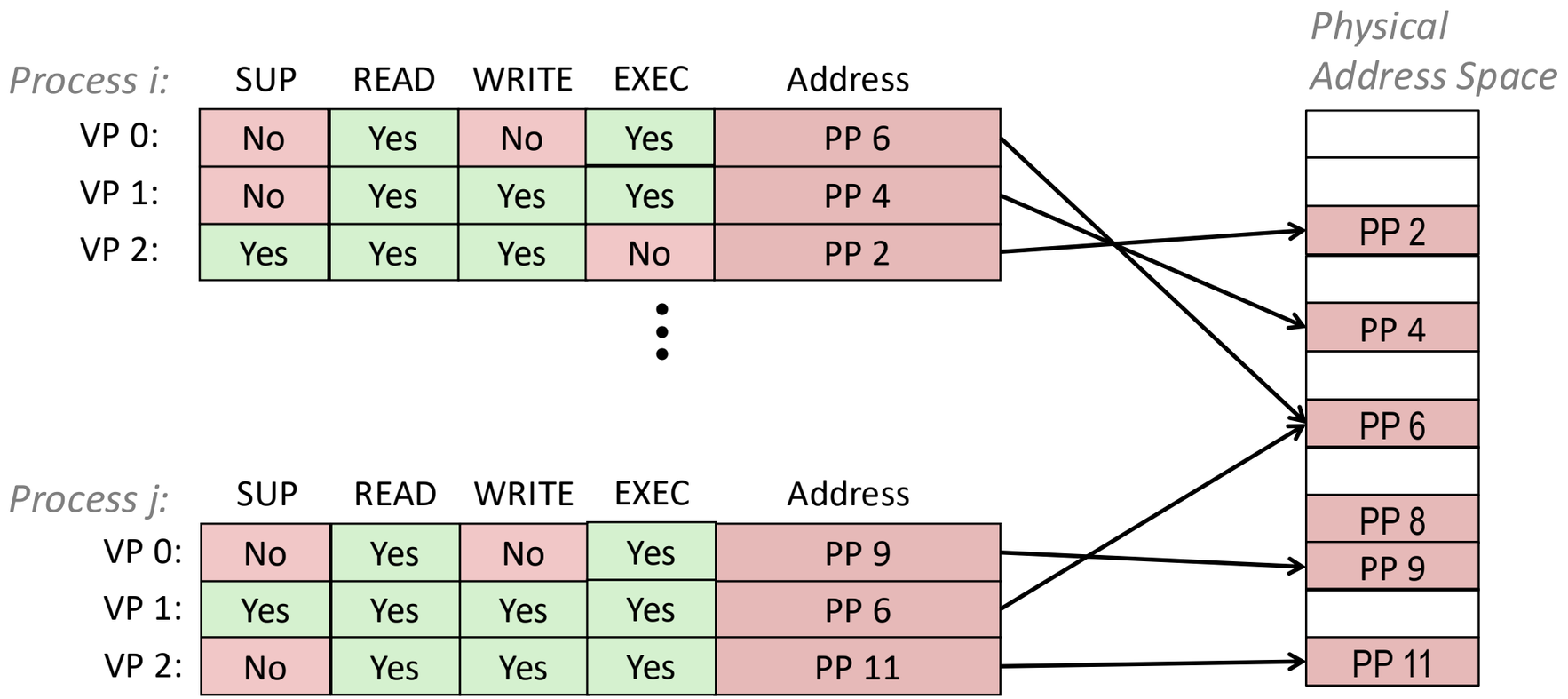
VM as a Tool for Memory Management

- Simplifying memory allocation
  - Each virtual page can be mapped to any physical page
  - A virtual page can be stored in different physical pages at different times
- Sharing code and data among processes
  - Map virtual pages to the same physical page (here: PP 6)



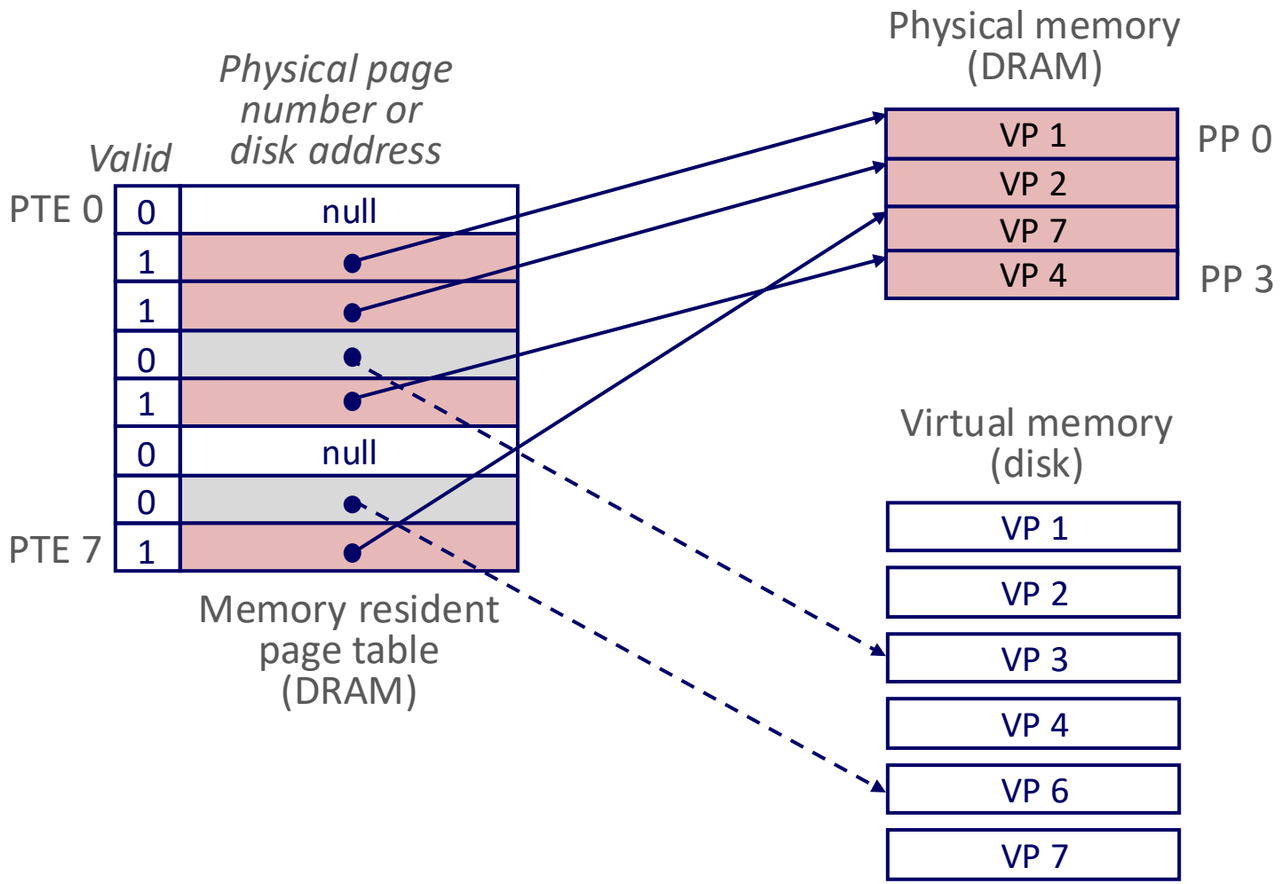
VM as a Tool for Memory Protection

- Extend PTEs with permission bits
- MMU checks these bits on each access



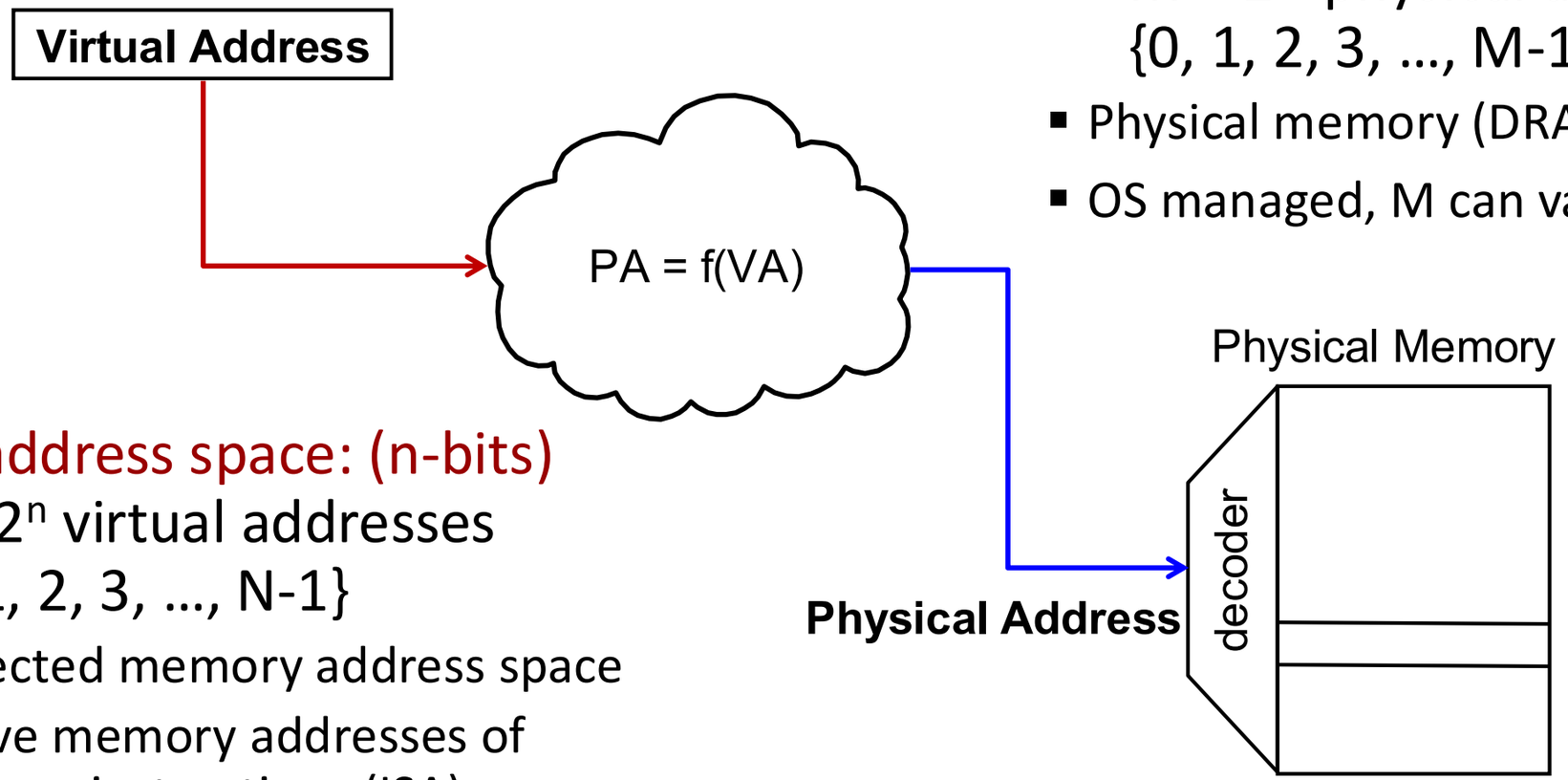
Enabling Data Structure: Page Table

- A *page table* is an array of page table entries (PTEs) that maps virtual pages to physical pages.
  - Per-process kernel data structure in DRAM



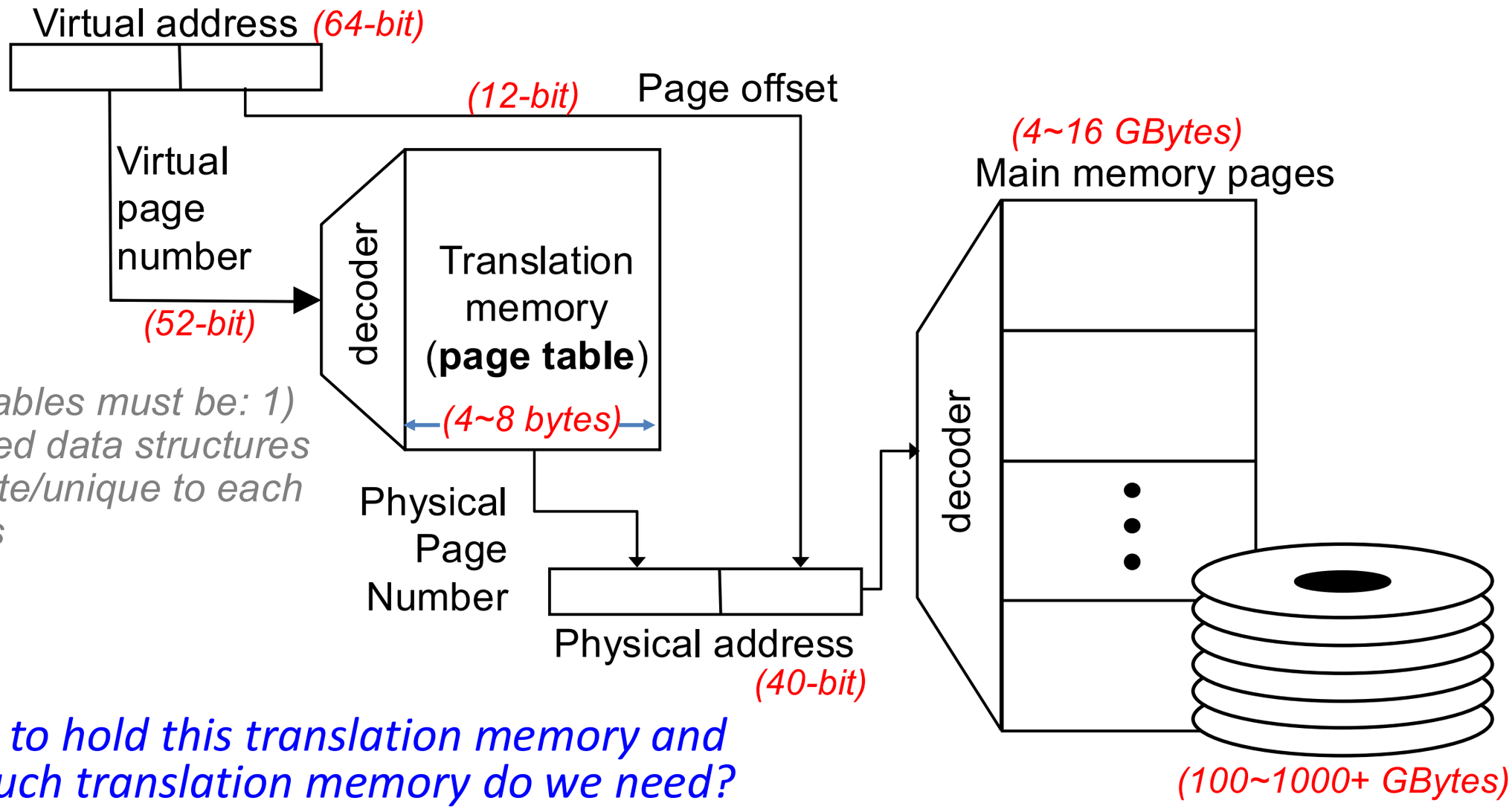
Virtual vs. Physical Address Spaces

- Physical address space: (m-bits)
  - $M = 2^m$ physical addresses
  - {0, 1, 2, 3, …, M-1}
  - Physical memory (DRAM) address space
  - OS managed, M can vary and M ≠ N



- Virtual address space: (n-bits)
  - $N = 2^n$ virtual addresses
  - {0, 1, 2, 3, …, N-1}
  - Architected memory address space
  - Effective memory addresses of load/store instructions (ISA)
  - Program view of memory addresses

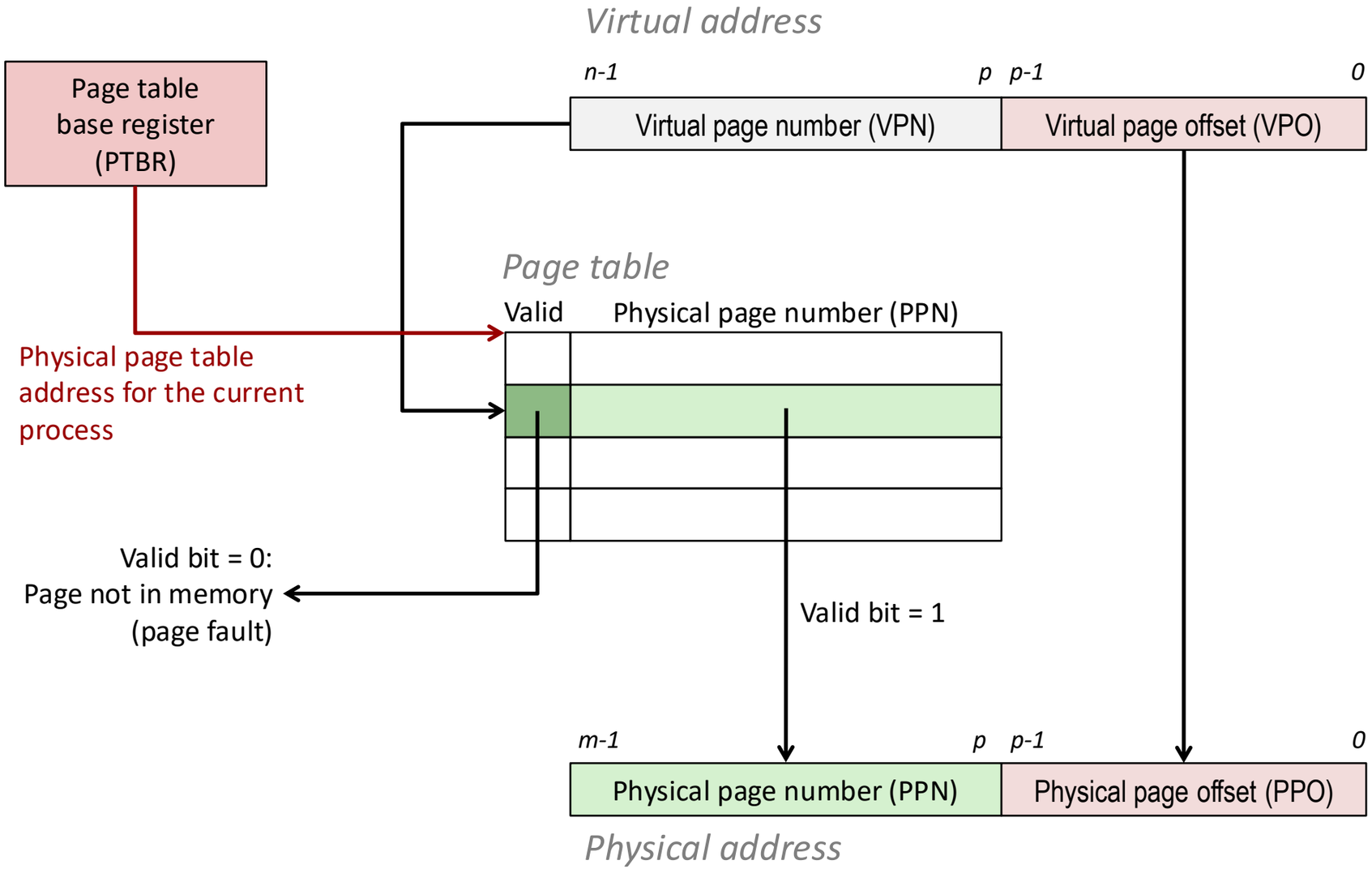
Page-Based (Paging) Virtual Memory



Page Tables must be: 1) privileged data structures 2) private/unique to each process

Where to hold this translation memory and how much translation memory do we need?

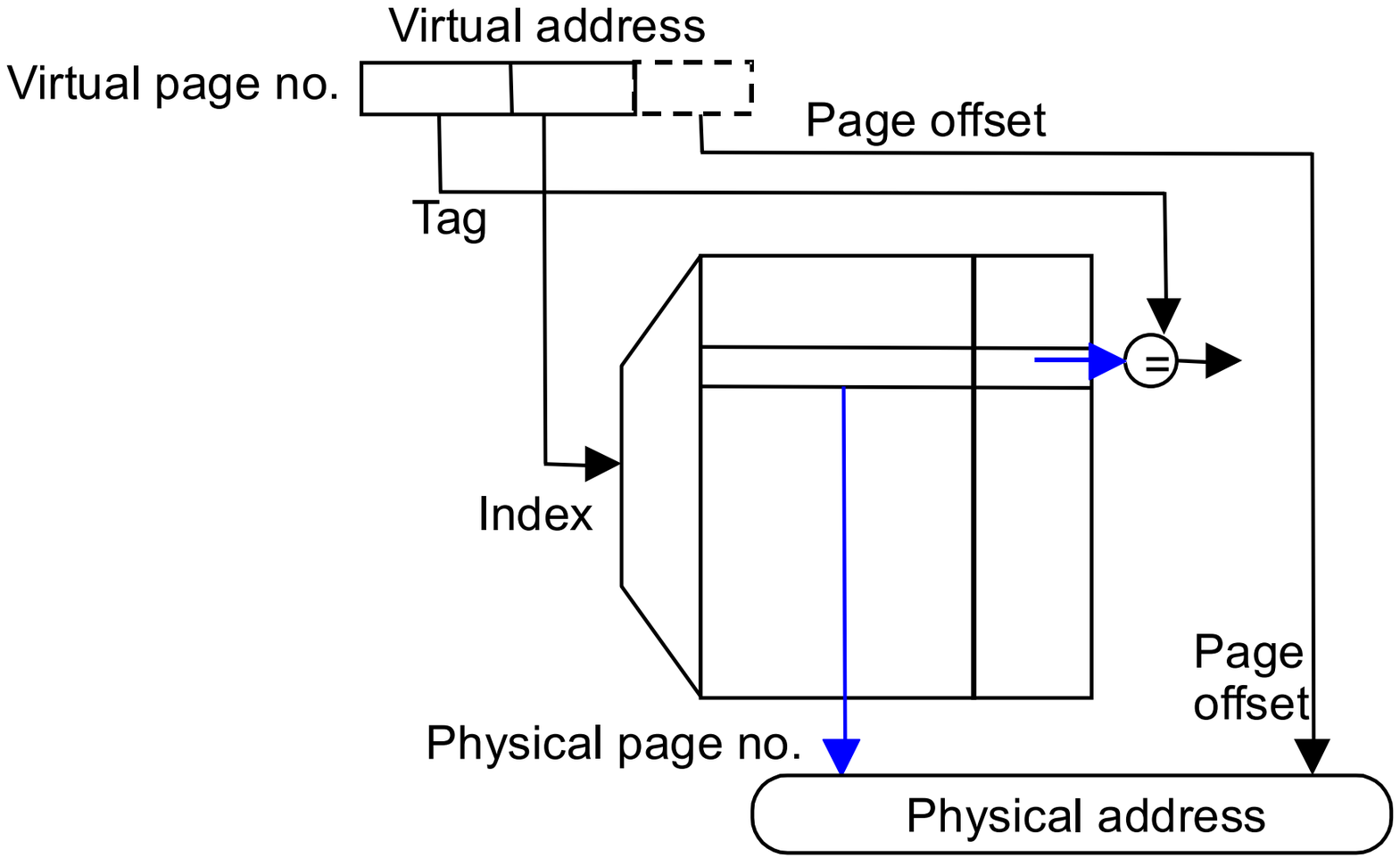
Address Translation With a Page Table



Speeding up Translation with a TLB

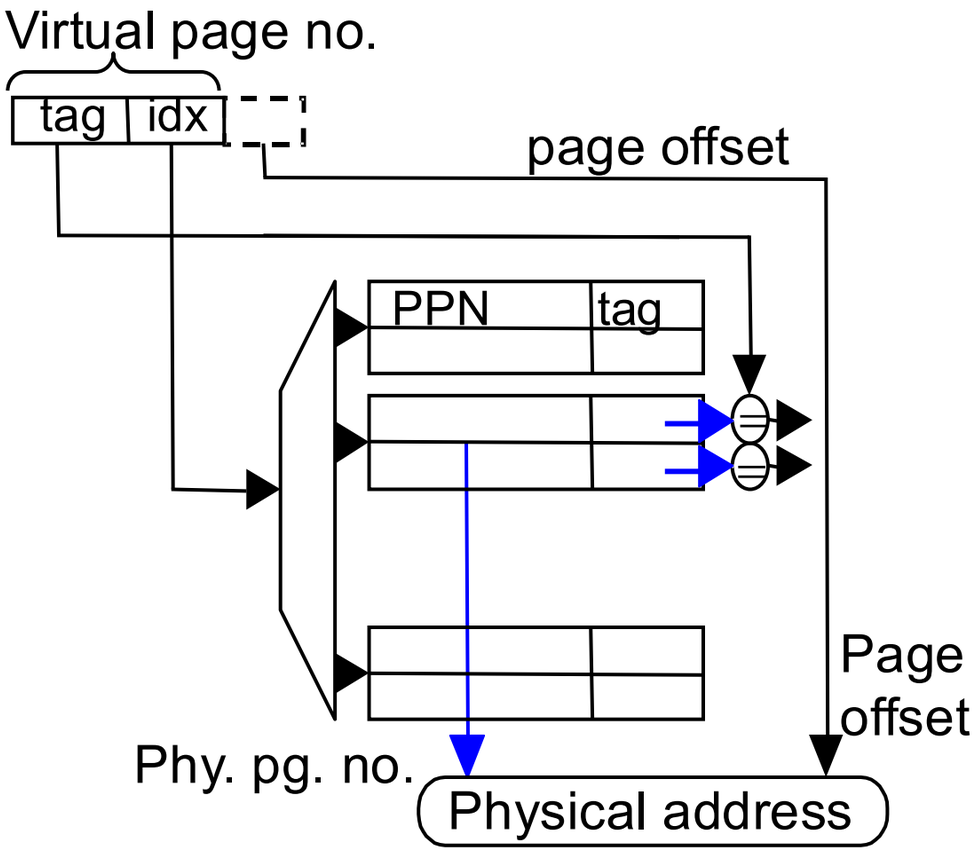
- **Page table entries (PTEs) are cached in L1 like any other memory word**
  - PTEs may be evicted by other data references
  - PTE hit still requires a small L1 delay
- **Solution: *Translation Lookaside Buffer* (TLB)**
  - Small set-associative hardware cache in MMU
  - Maps virtual page numbers to physical page numbers
  - Contains complete page table entries for small number of pages

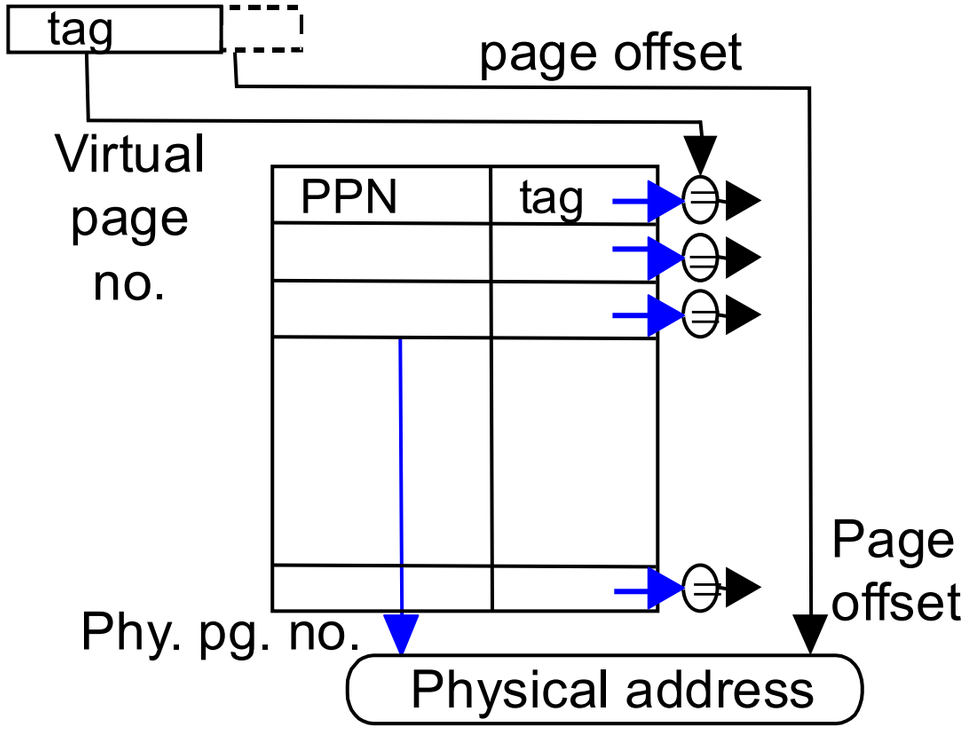
Translation Look-aside Buffer (TLB)



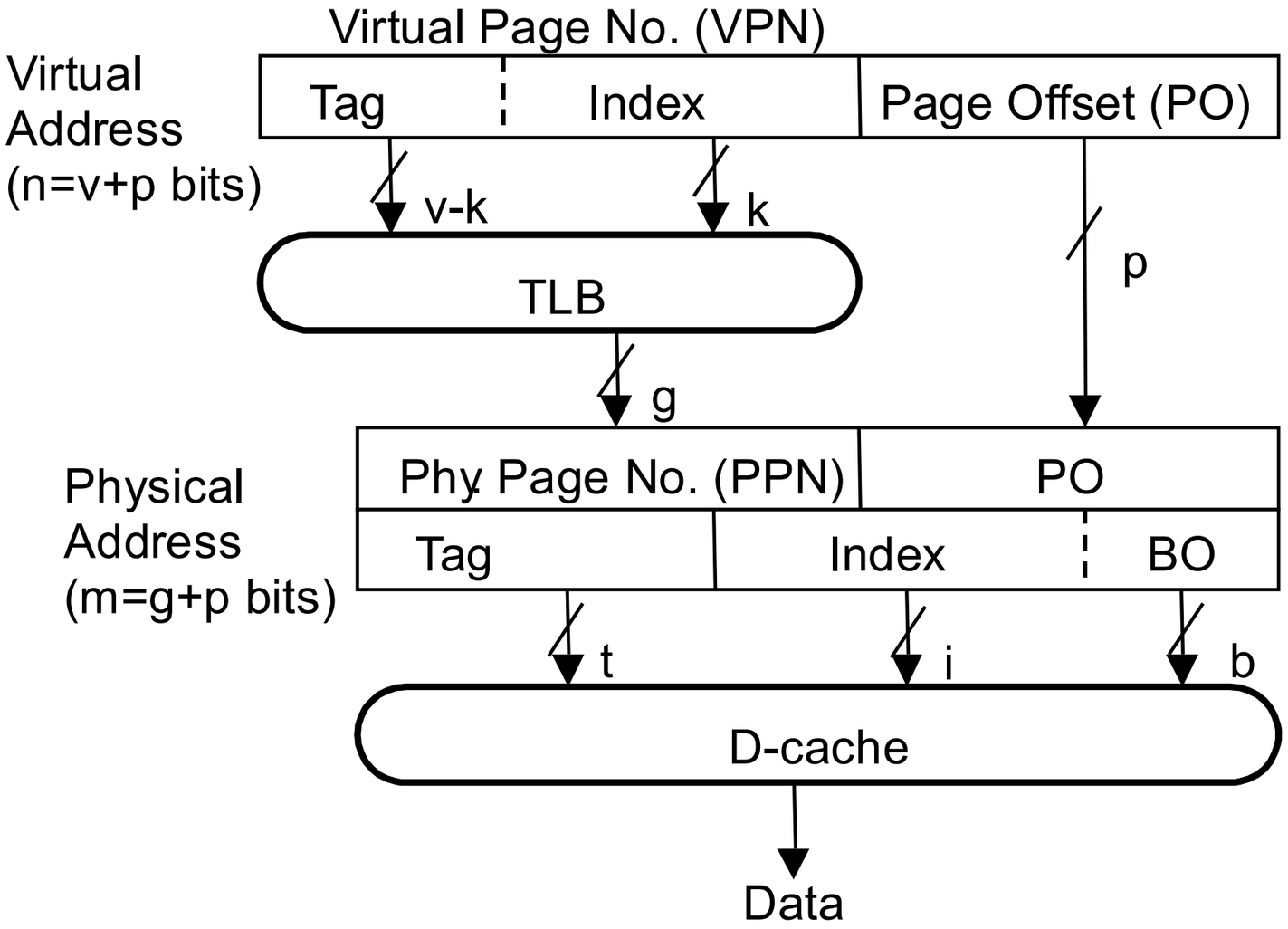
A cache for the page table for fast address translations

Set-Associative and Fully Associative TLBs

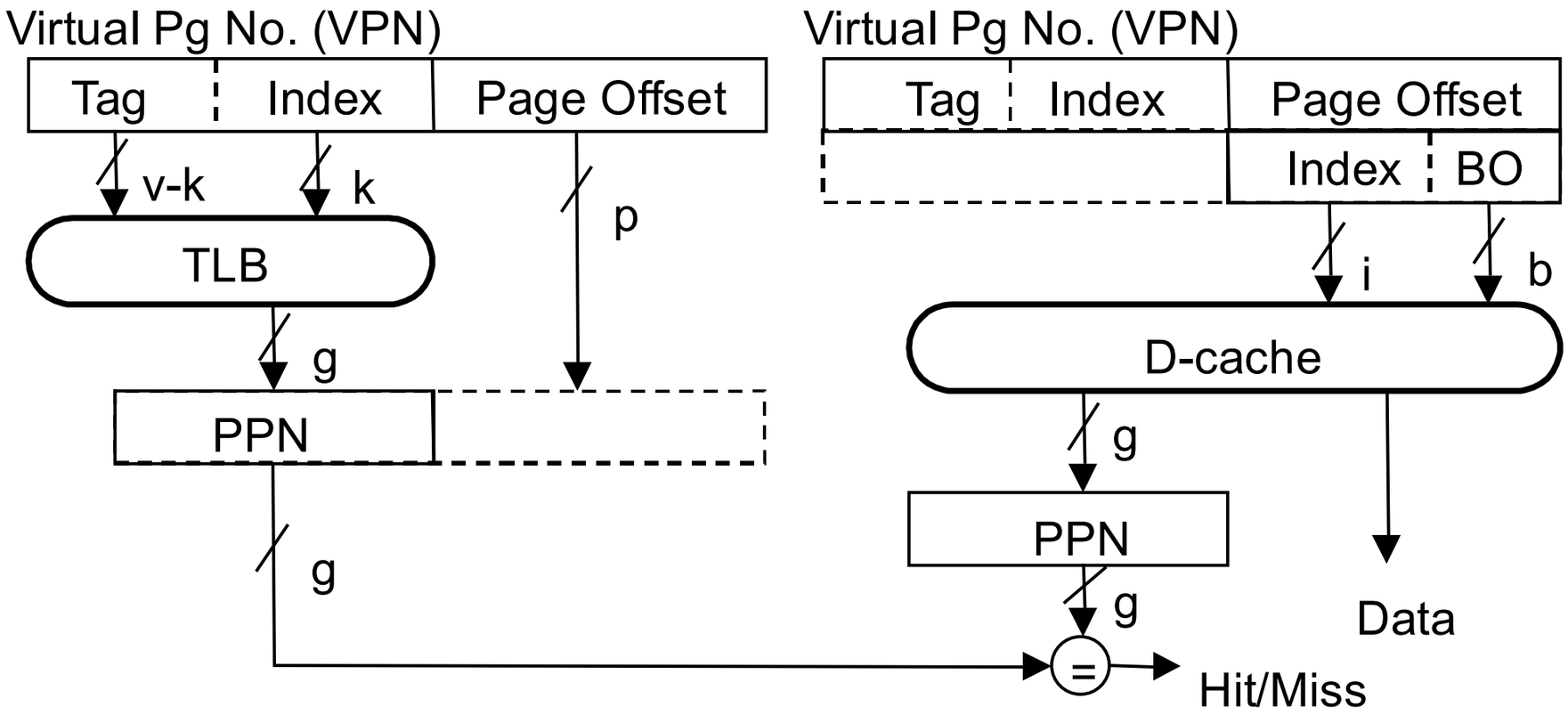




Physically Indexed Cache



Virtually Indexed Cache

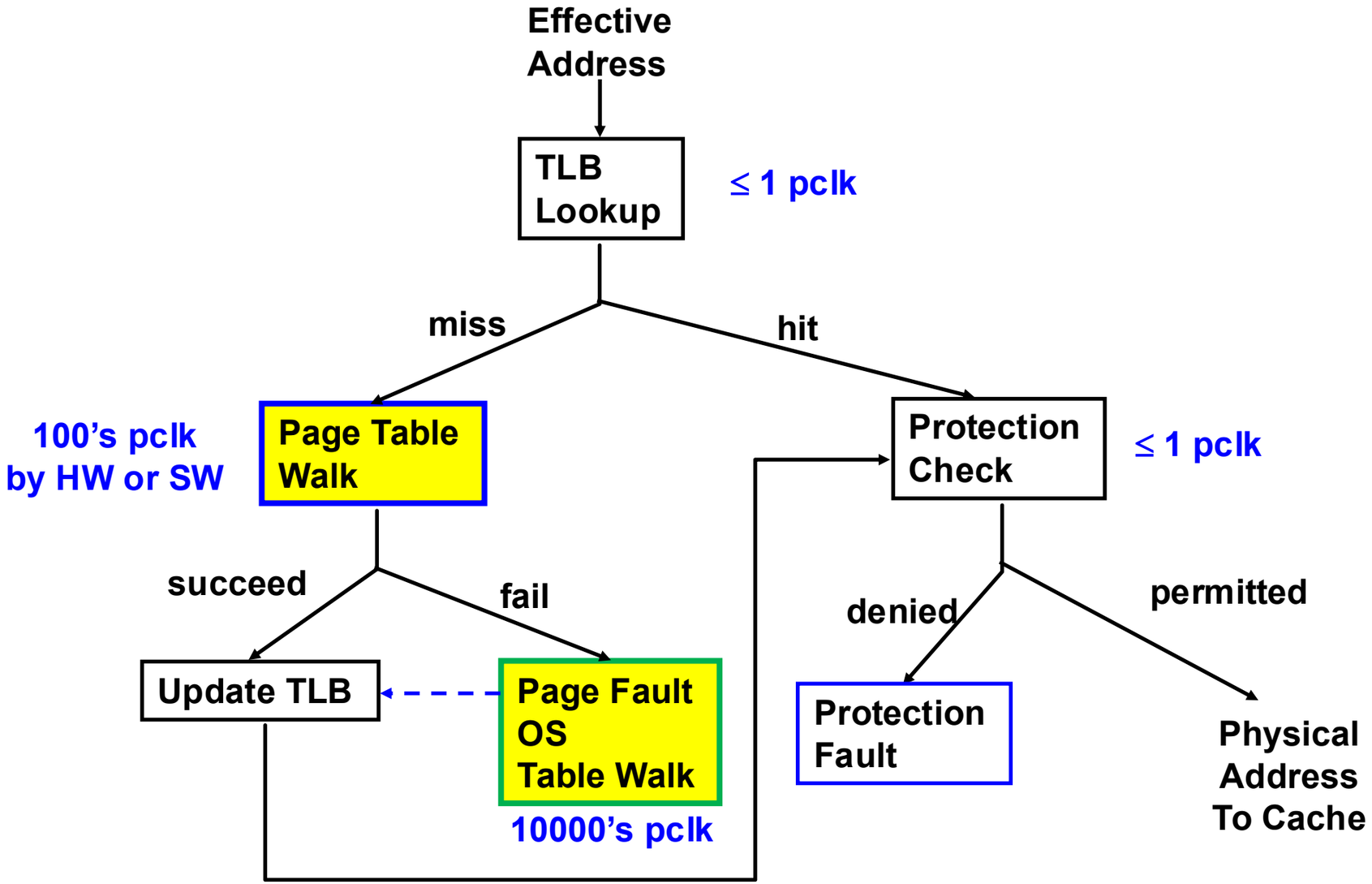


How big can a virtually indexed cache get?

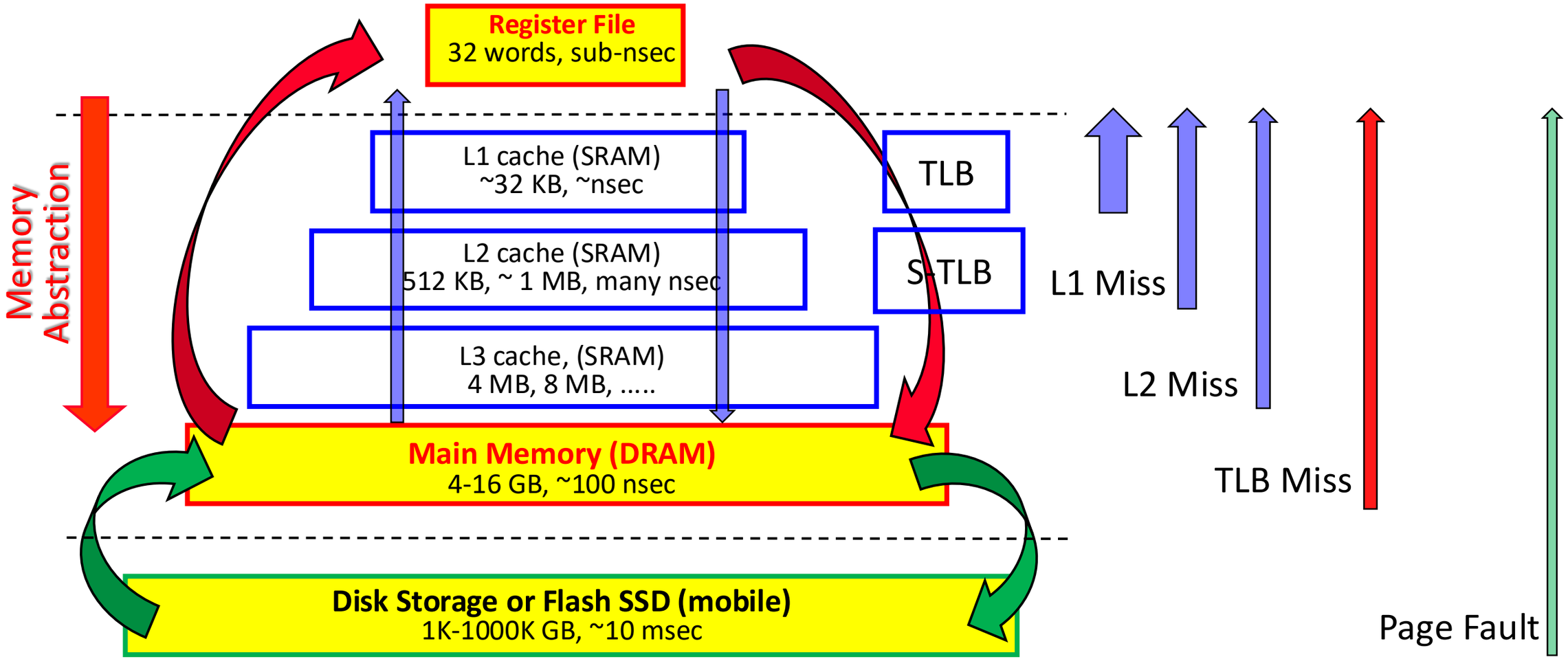
Summary of Address Translation Symbols

- Basic Parameters
  - **N = 2^n** : Number of addresses in virtual address space
  - **M = 2^m** : Number of addresses in physical address space
  - **P = 2^p** : Page size (bytes)
- Components of the virtual address (VA)
  - **TLBI**: TLB index
  - **TLBT**: TLB tag
  - **VPO**: Virtual page offset
  - **VPN**: Virtual page number
- Components of the physical address (PA)
  - **PPO**: Physical page offset (same as VPO)
  - **PPN:** Physical page number
  - **CO**: Byte offset within cache line
  - **CI:** Cache index
  - **CT**: Cache tag

Virtual to Physical Address Translation



Memory Hierarchy Dynamic Behaviors



18-740: Modern Computer Architecture & Design

Lecture 7:
"Dealing with Long-Latency Memory Instructions"

A. Virtual Memory Revisited (18-213/18-613)

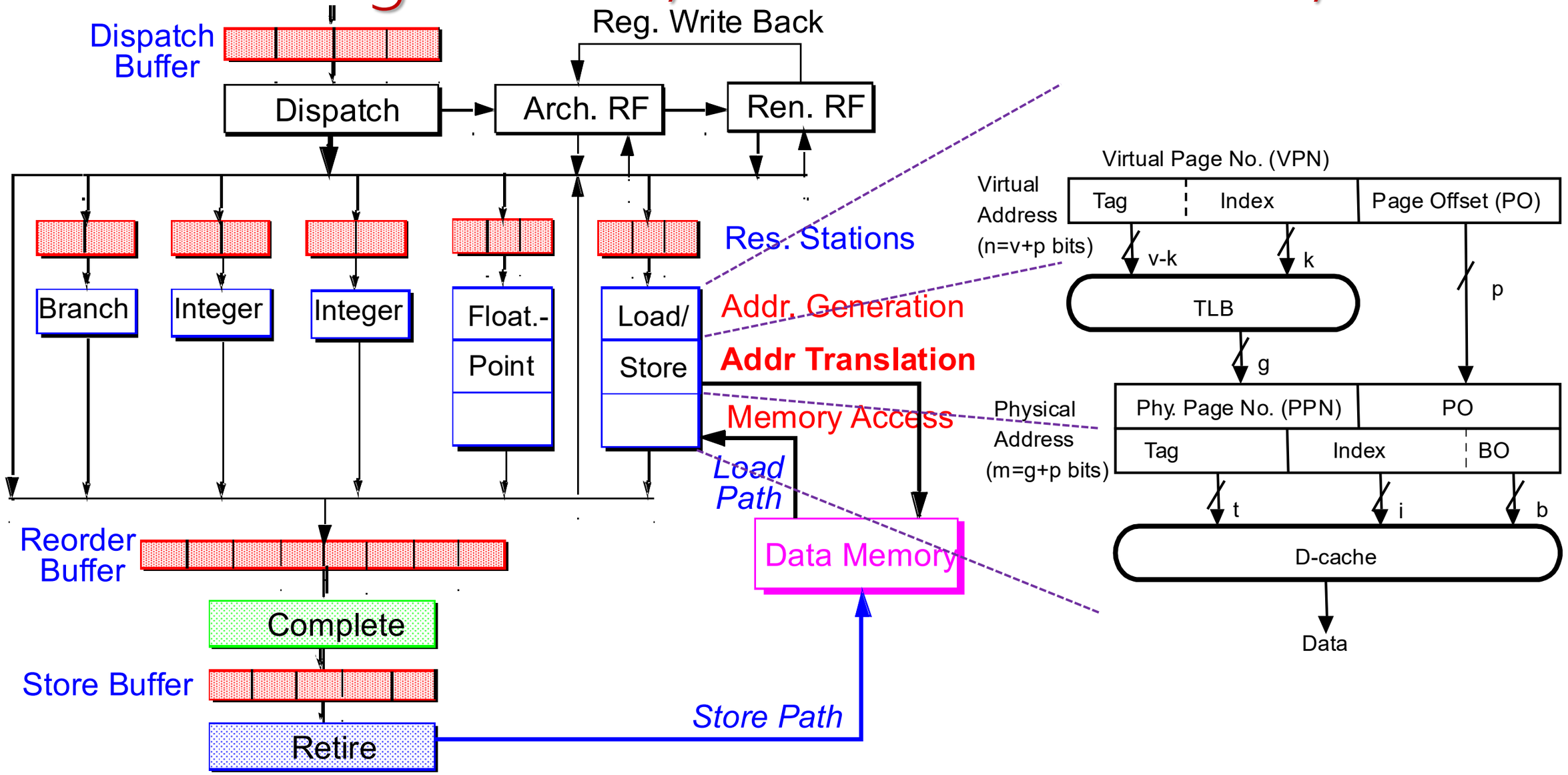
B. Memory Operations and Dataflow

C. Modern Superscalar Out-of-Order Processors (ARM CPUs)

D. Snapdragon 8 Gen 2 + ARM Cortex A715 & X3 CPU Cores

Electrical & Computer ENGINEERING

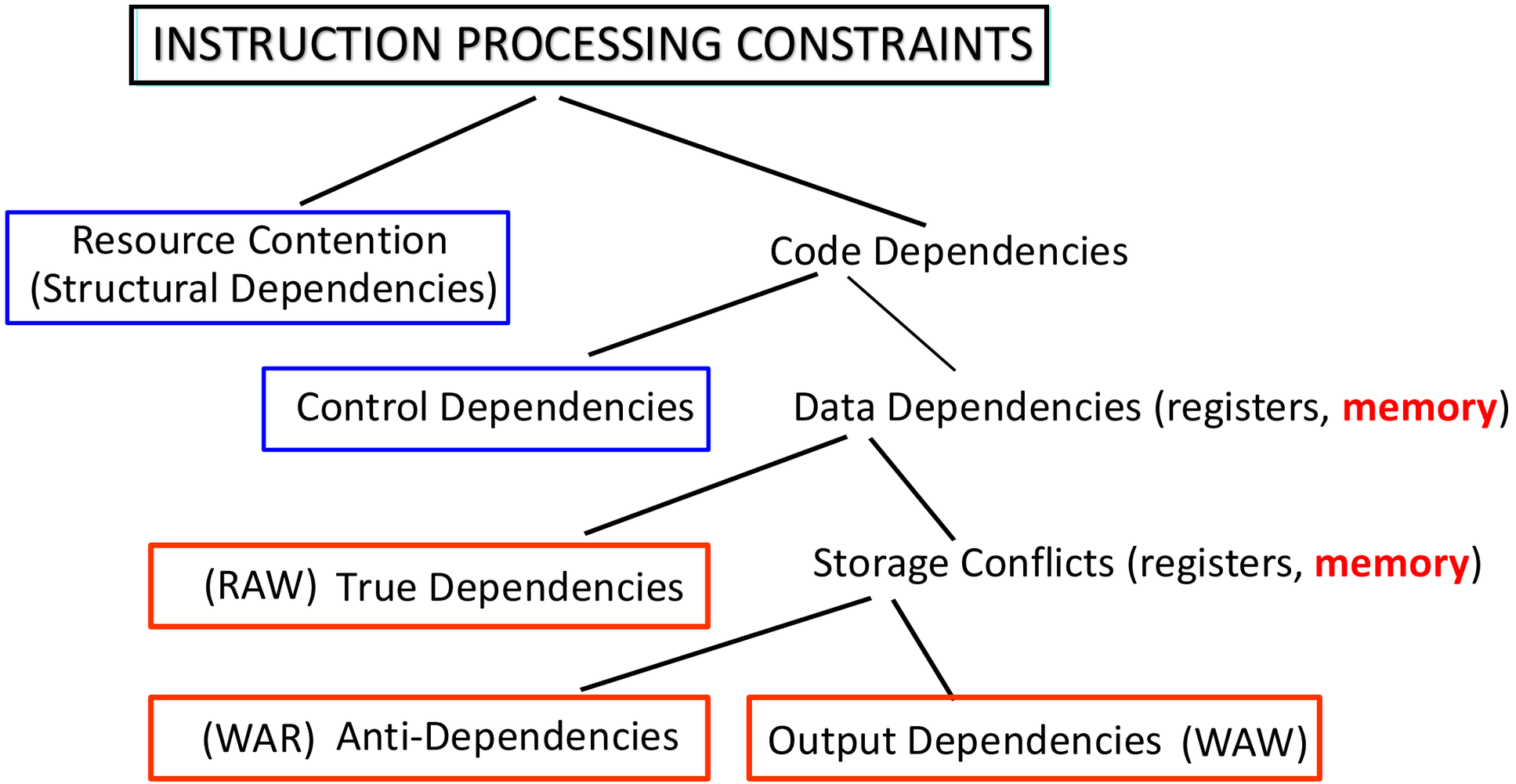
Processing of Load/Store Instructions w/ VM



Memory Operations and Data Flow

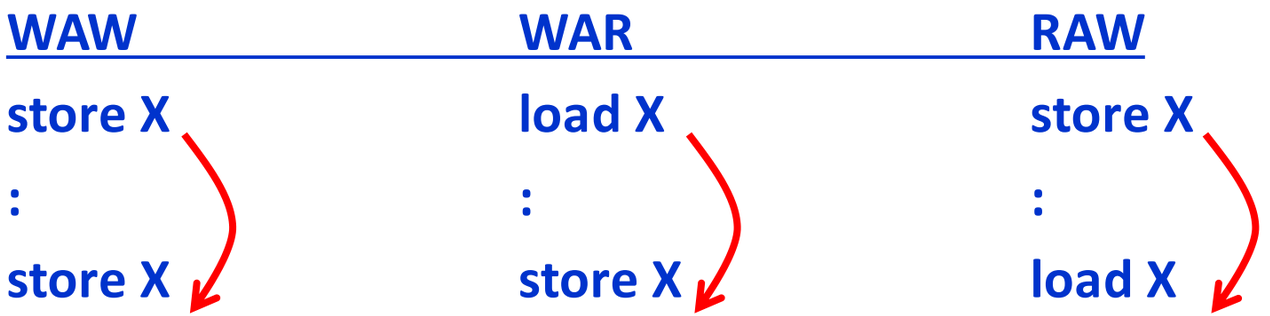
- So far, we have only considered register-register instructions
  - Add, sub, mul, branch, jump,
- Loads and Stores
  - Necessary because we don't have enough registers for everything
    - Memory allocated objects, register spill code
  - RISC ISAs: only loads and stores access memory
  - CISC ISAs: memory micro-ops are essentially RISC loads/stores
- Steps in load/store processing
  - Generate address (not fully encoded by instruction)
  - Translate address (virtual ⇒ physical)
  - Execute memory access (actual load/store)

The Big Picture: Impediments Limiting ILP



A. Memory Data Dependencies

- Besides branches, long memory latencies are one of the biggest performance challenges today.
- To preserve sequential (in-order) state in the data caches and external memory (so that recovery from exceptions is possible) stores are performed in order. This takes care of anti-dependencies and output dependencies to memory locations.
- However, loads can be issued out of order with respect to stores if the out-of-order loads check for data dependencies with respect to previous, pending stores.

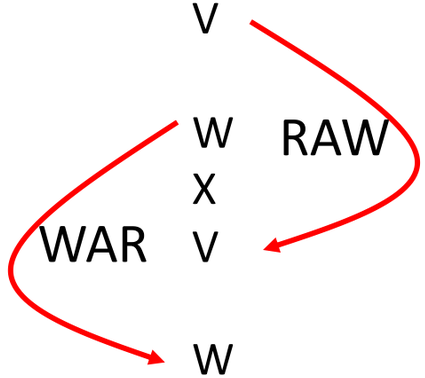


Memory Data Dependency Terminology

- "**Memory Aliasing**" = Two memory references involving the same memory location (collision of two memory addresses).
- "**Memory Disambiguation**" = Determine whether two memory references will alias or not (whether there is a dependency or not).
- **Memory Dependency Detection**:
  - Must compute effective addresses of both memory references
  - Effective addresses can depend on run-time data and other instructions
  - Comparison of addresses require much wider comparators

Example code:

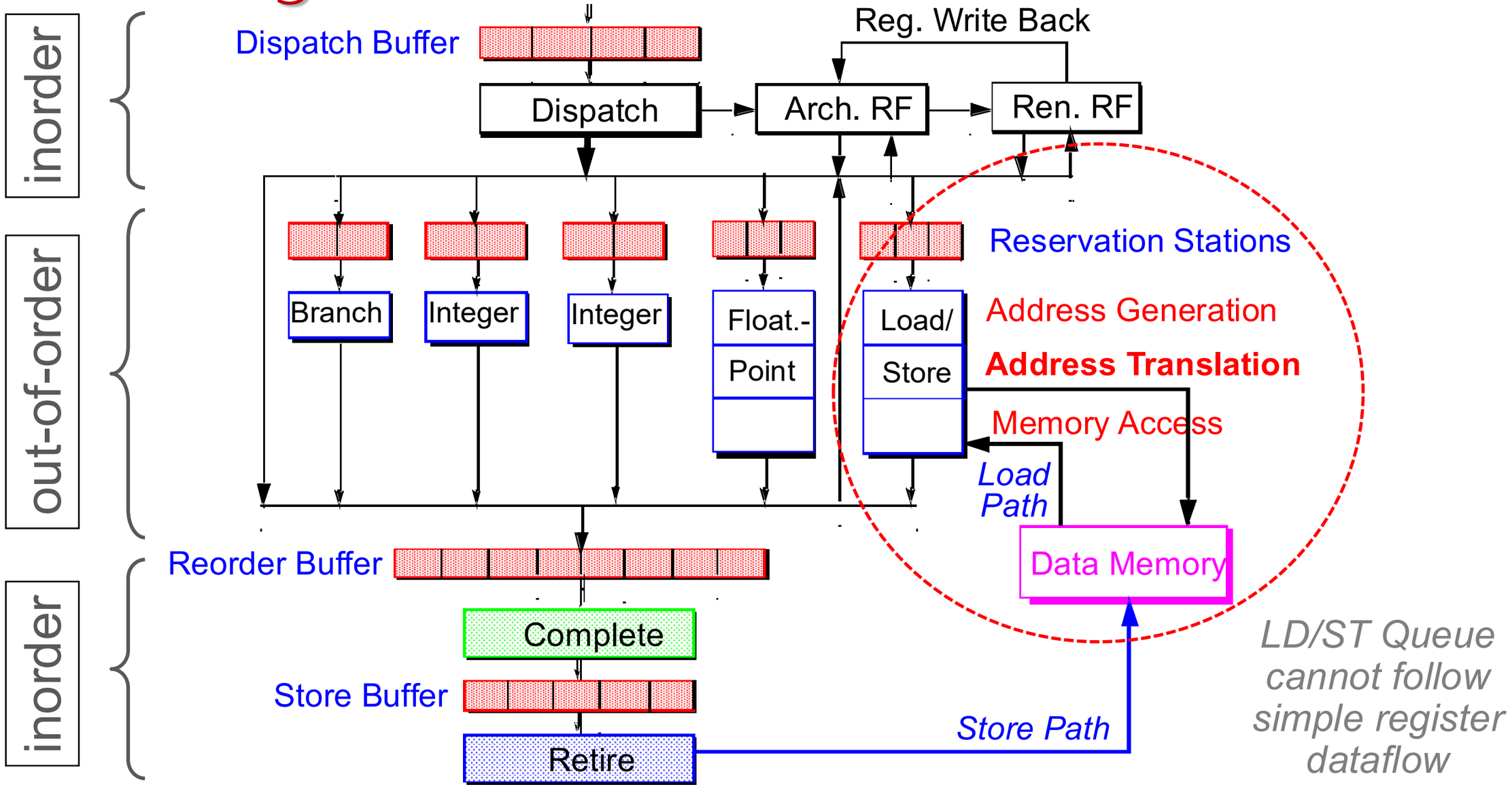
```
(1)   STORE   V
(2)   ADD
(3)   LOAD    W
(4)   LOAD    X
(5)   LOAD    V
(6)   ADD
(7)   STORE   W
```



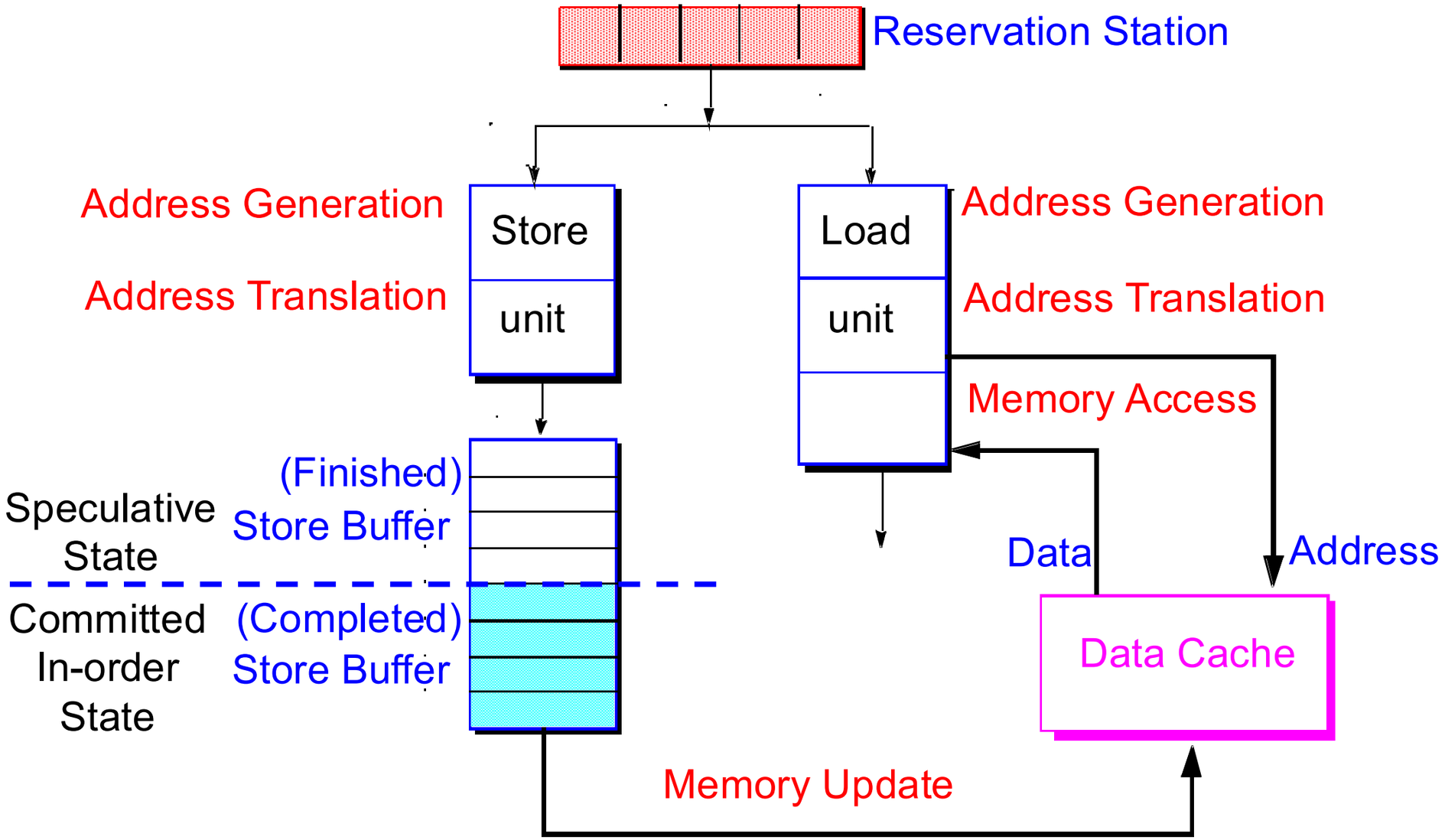
Total Ordering of Loads and Stores

- Keep all loads and stores totally in order with respect to each other.
- However, loads and stores can execute out of order with respect to other types of instructions (while obeying register data dependencies).
- Consequently, stores are held for all previous instructions, and loads are held for stores.
  - I.e. stores performed at commit point
  - Sufficient to prevent wrong branch path stores since all prior branches now resolved

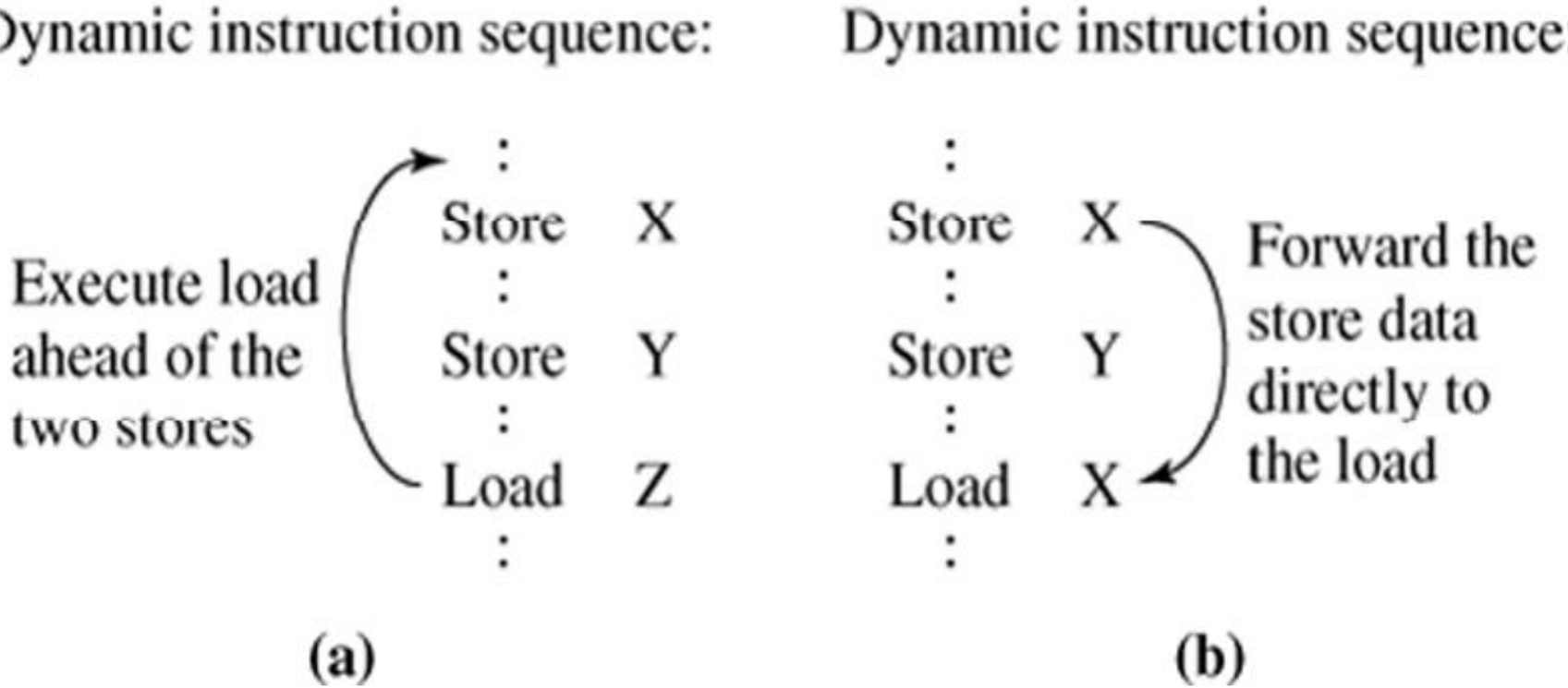
Processing of Load/Store Instructions



Load/Store Units and Store Buffer



Load Bypassing & Load Forwarding: Motivation

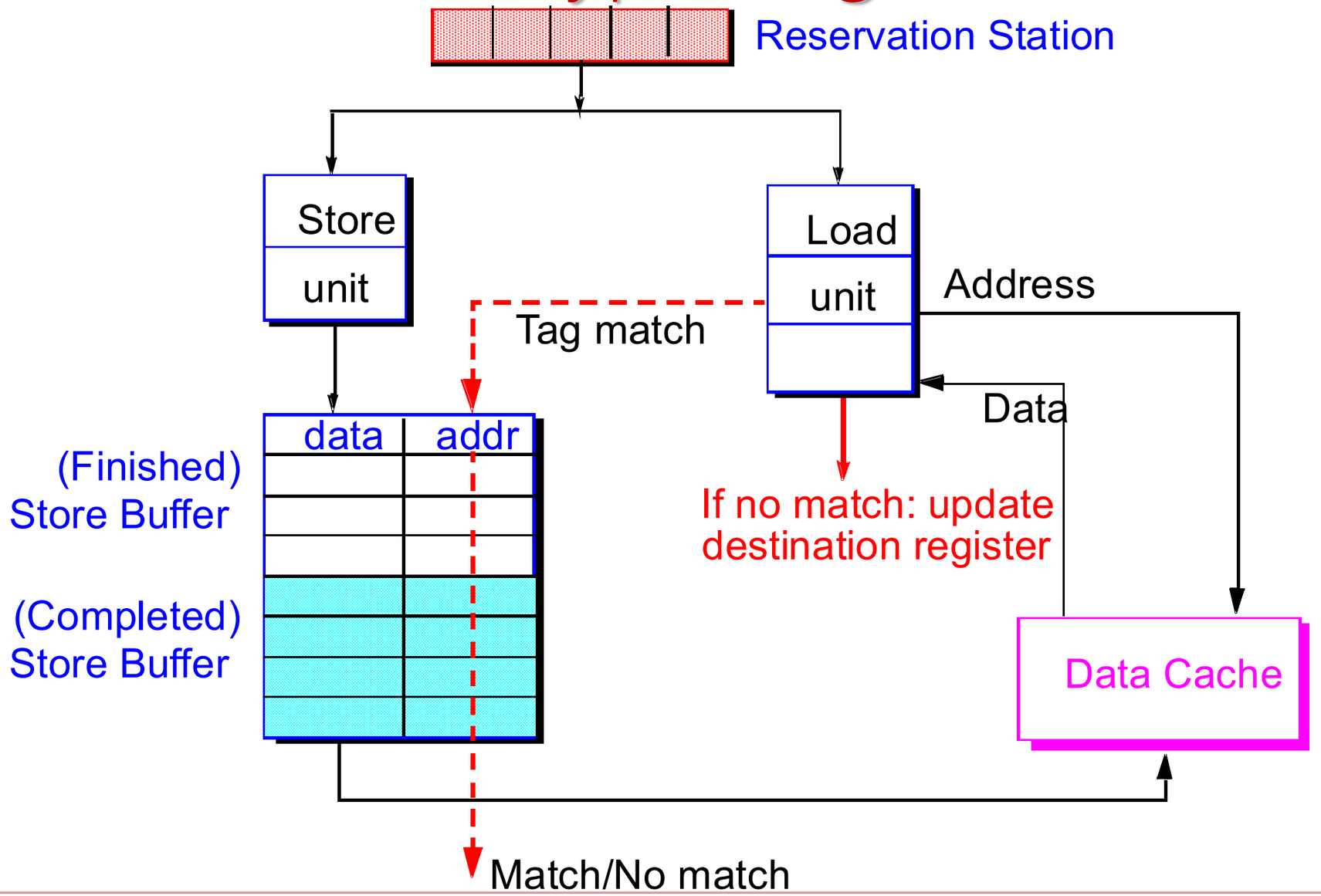


B. Load Bypassing

- Loads can be allowed to bypass older stores if no aliasing is found
  - Older stores' addresses must be computed before loads can be issued to allow checking for RAW load dependences. If dependence cannot be checked, e.g. store address cannot be determined, then all subsequent loads are held until address is valid (conservative).
- Alternatively a load can assume no aliasing and bypass older stores *speculatively*
  - Validation of no aliasing with previous stores must be done and mechanism for reversing the effect must be provided.
- Stores are kept in ROB until all previous instructions complete, and kept in the store buffer until gaining access to cache port.
  - At completion time, a store is moved to the Completed Store Buffer to wait for turn to access cache. Store buffer is "future file" for memory.

Store is consider completed. Latency beyond this point has little effect on the processor throughput.

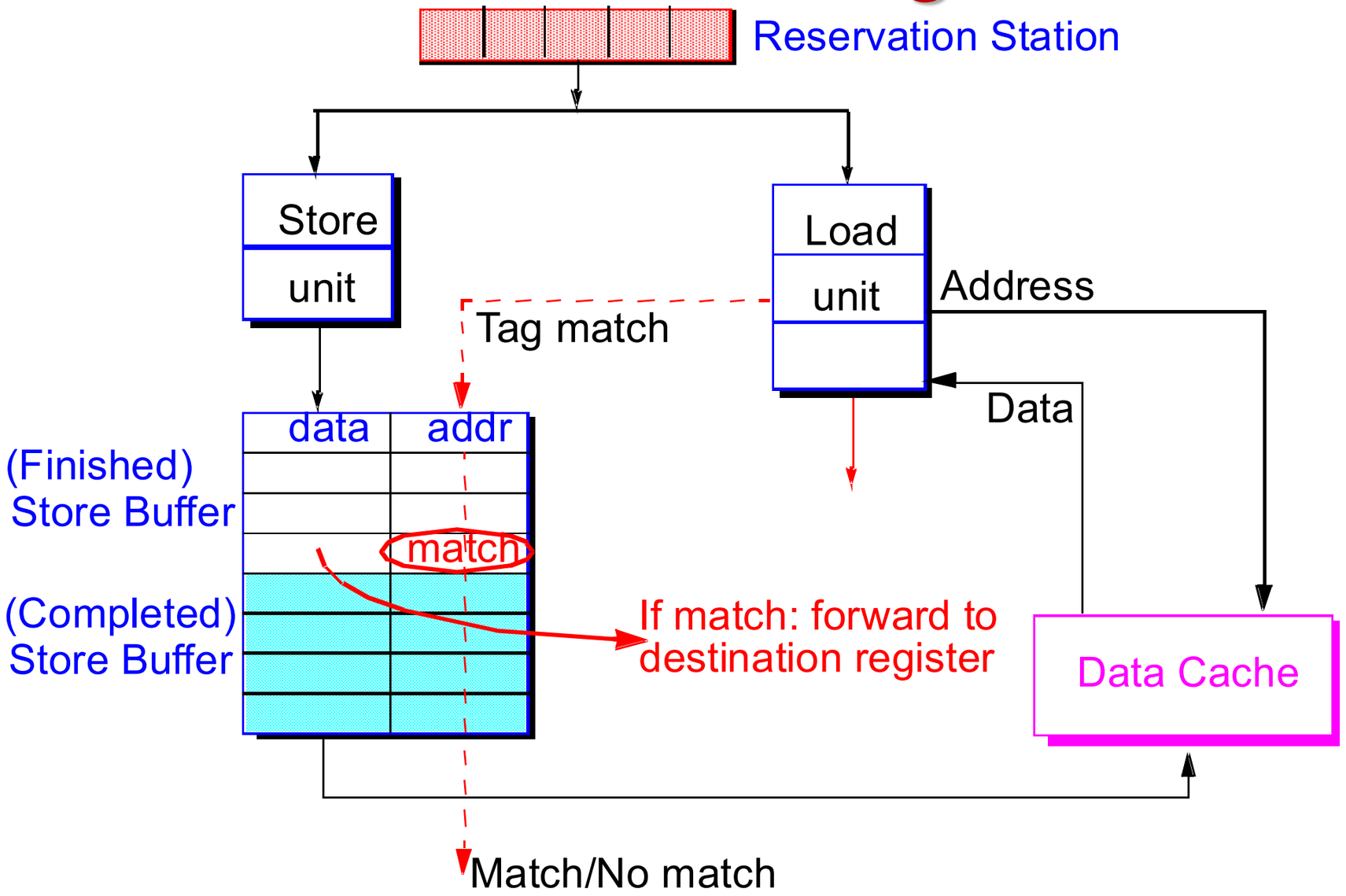
Illustration of Load Bypassing



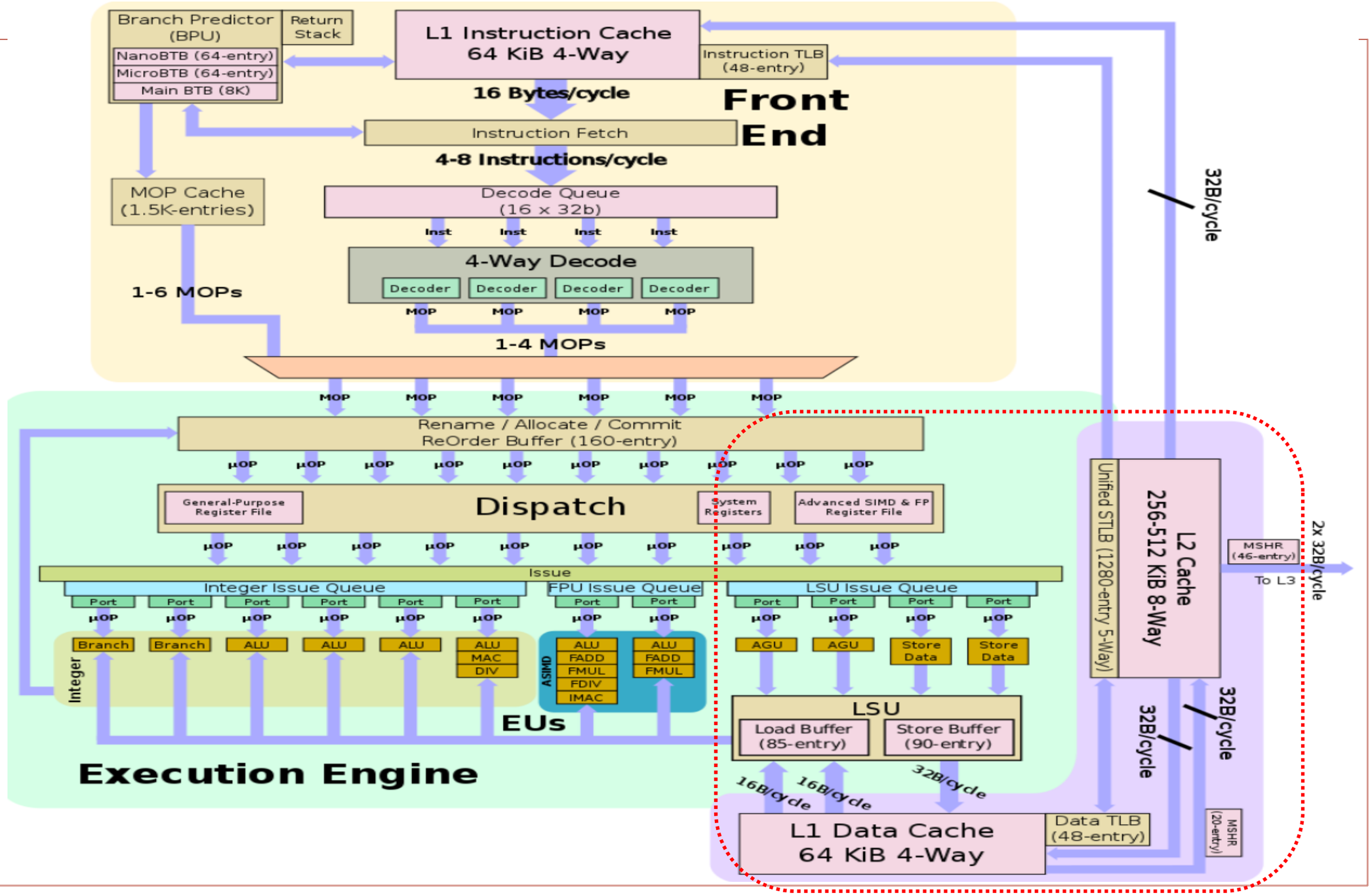
C. Load Forwarding

- If a pending load is RAW dependent on an earlier store still in the store buffer, it need not wait till the store is issued to the data cache
- The load can be directly satisfied from the store buffer if both load and store addresses are valid and the data is available in the store buffer
- Since data is sourced directly from the store buffer, this avoids the latency (and power consumption) of accessing the data cache

Illustration of Load Forwarding



Arm's New Cortex-A77 CPU Micro-architecture:



18-740: Modern Computer Architecture & Design

Lecture 8:
"Modern Memory Hierarchy and Data Storage"

Olivia Hsu

September 17, 2026

Next Time ...

- Required Reading Assignment:
  - **Chapter 3 of Shen and Lipasti (SnL).**
  - **From Synthesis lectures of Computer Architecture**
    - Sec. 1 & 3: Bruce Jacob, "The Memory System: You Can't Avoid It, You Can't Ignore It, You Can't Fake It," 2009.
    - Sec. 1 & 4: Balasubramonian Rajeev, "Innovations in the Memory System," 2019.
- **Lab 2** due 10/9/2026, start early!
- **HW 1** due 9/18/2026

Electrical & Computer ENGINEERING